Pattern Recognition 78 (2018) 56–78

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/patcog

Infinite Bayesian one-class support vector machine based on Dirichlet process mixture clustering

Wei Zhang[a,b], Lan Du[a,b,*], Liling Li[a,b], Xuefeng Zhang[a,b], Hongwei Liu[a,b]

[a] National Lab of Radar Signal Processing, Xidian University, Xi'an 710071, China
[b] Collaborative Innovation Center of Information Sensing and Understanding at Xidian University, Xi'an 710071, China

ARTICLE INFO

Article history:
Received 1 August 2016
Revised 11 September 2017
Accepted 7 January 2018
Available online 11 January 2018

Keywords:
Dirichlet process mixture
One-class classifiers
One-class support vector machine
Gibbs sampling

ABSTRACT

In the problem of one-class classification, one-class classifier (OCC) tries to identify objects of a specific class, called the target class, among all objects, by learning from a training set containing only the objects of target class. However, some traditional OCCs lose their effectiveness when the distribution of target class is a multimodal distribution, and the performance is sensitive to the model parameters. To solve this problem and enhance classification performance, a novel OCC infinite Bayesian OCSVM (IB-OCSVM) is proposed in this study. In IB-OCSVM, we partition the input data into several clusters with the Dirichlet process mixture (DPM), and learn a modified OCSVM in each cluster. Specifically, the clustering procedure and the modified OCSVM are jointly learned in a unified Bayesian frame to guarantee the consistency of clustering and linear separability in each cluster. The parameters can be inferred simply and effectively via Gibbs sampling technique. Meanwhile, we modify the traditional OCSVM by replacing the kernel transformation with a new feature transformation based on the result of clustering via DPM. In the feature space of the modified OCSVM, all transformed samples preserve the same clustering characteristic as the original samples, thus guaranteeing the combination of clustering and classification in our method. Experimental results based on benchmark datasets and synthetic aperture radar real data demonstrate that the proposed method has enhanced classification performance and is more robust to the model parameters than some conventional methods.

1. Introduction

The one-class classification problem, also known as outlier or novelty/anomaly detection, is concerned with identifying objects of a specific class, termed target class, among all objects. The task of one-class classification is to build a boundary around the target class by learning from a training set containing only the objects of the target class. In addition, one-class classification problem is different from and more difficult than binary-class classification problem, in which objects from the two classes are available and the decision boundary is supported by the presence of samples from each class [1,2].

The interest in one-class classification problem is both methodological and of an application-oriented nature [3–18]. Several effective methods proposed during the past years have achieved good performance in some applications. For example, in one-class support vector machine (OCSVM) [26], hyperplane is constructed in the feature space, and a sample is an outlier if it's located below the hyperplane; in k-means clustering [28], a sample is an outlier if the minimum distance from the cluster centres of target class is larger than a threshold. However, some traditional OCCs lose their effectiveness when the distribution of target class is a multimodal distribution, and the performance is sensitive to the model parameters.

The current study proposes a novel OCC based on the Dirichlet process mixture (DPM) clustering and modified OCSVM, referred to as the infinite Bayesian OCSVM (IB-OCSVM), which is robust to the parameter used in the feature transformation and can obtain enhanced classification performance compared with the traditional methods.

In IB-OCSVM, we partition the input data into several clusters with DPM, and learn a modified OCSVM in each cluster. Meanwhile, in the modified OCSVM, we replace the kernel transformation with a novel feature transformation based on the clustering result of DPM.

The main contributions of this study are summarized in the following:

- Our IB-OCSVM is a combination of the DPM clustering and the modified OCSVM. Specifically, the clustering procedure and the

* Corresponding author at: 2 Taibai Rd., Xi'an, Shaanxi 710071, China.
E-mail addresses: dulan@mail.xidian.edu.cn, orchid_du@163.com (L. Du).

https://doi.org/10.1016/j.patcog.2018.01.006
0031-3203/

modified OCSVM are jointly learned in a unified Bayesian frame to guarantee the consistency of clustering and linear separability in each cluster.

- A new feature transformation is proposed to modify the traditional OCSVM. If the samples are close together in the original space, then so should be these samples in the feature space with the proposed feature transformation method. Therefore, In the feature space of the modified OCSVM, all transformed samples preserve the same clustering characteristic as the original samples in the original data space, guaranteeing the combination of clustering and classification in our method. The transformed samples also lie within the hypersquare in the first quadrant with the negative samples scattering around the origin while the positive samples scatter away from the origin in the feature space, thus satisfying the requirement for feature distribution in the OCSVM.
- In our IB-OCSVM, the complex nonlinear classification can be implemented with simple linear classifiers in the feature space. One advantage of this structure is improving the robustness of the classification performance. Different from the traditional OCSVM that requires all kernel transformed features to be linearly separable, our IB-OCSVM only requires linear separability in each cluster in the feature space. Therefore, our model is less sensitive to the model parameter used in the new feature transformation. Moreover, the training data are clustered with DMP, an infinite mixture-of-expert model, which can automatically determine the number of clusters to solve the difficulty in obtaining suitable prior information about data in k-means clustering.

The remainder of the paper is organized as follows. Section 2 is a brief overview of related work and background material. Section 3 introduces the complete algorithm about IB-OCSVM. Section 4 validates the performance of our method on synthetic data, benchmark datasets, and synthetic aperture radar (SAR) real data. In Section 5, a summary of our method is given.

2. Related work and background material

In this Section, we give an introduction about the related work. Firstly, an overview on one-class classification methods is given. Then, OCSVM and DPM, the basis of our method, are introduced in detail, which helps authors understand the proposed method more easily.

2.1. Review on one-class classification methods

Tax et al. group most OCCs into three categories: density estimation methods, boundary methods, and reconstruction methods [1].

Density estimation methods estimate the probability density function of the training data and set a threshold on this density. The most commonly used distribution for continuous variables is the Gaussian distribution [2]. More complex data can be modeled by mixture models, such as Gaussian mixture model [19,20], or mixtures of other types of distributions such as gamma distribution [21,22], Poisson distribution [23], Student's distribution [24], and Weibull distribution [25].

Boundary-based methods create a boundary based on the structure of the training dataset. Support vector data description (SVDD) [4] and OCSVM [26] are two classical bounding-based methods. The idea of OCSVM is to form a hyperplane in the feature space, by separating the transformed data from the origin with the maximum margin. SVDD constructs a hypersphere with minimum volume that encloses all (or most) of the training data. Pekalska proposes a novel boundary-based method, namely linear programming data description (LPDD), in [9]. With linear programming, LPDD can find the general dissimilarity representation and build a classification hyperplane in the dissimilarity space. Lanckriet introduces a boundary-based method, the minimax probability machine (MPM), for novelty detection in [27]. MPM tries to construct a hyperplane that separates the data from the origin, by minimizing the maximal probability of misclassification of the target data in all possible class-conditional densities with a given mean and covariance matrix. Other more recent boundary-based methods include position regularized SVDD [10], prototype-based domain description [12], density-induced SVDD [13], and weighted OCSVM [16].

For most reconstruction-based methods, a representation model or a set of prototypes is learned from the target data by minimizing the reconstruction error. When the reconstruction error of a test sample is lower than a threshold, the sample belongs to the target class; otherwise, it is an outlier. k-means clustering [28] is the simplest reconstruction-based method. In k-means clustering, the target data can be characterized by a few prototype objects and the distance of the test sample to the nearest prototype is used for comparison with the threshold. The method based on auto-encoder neural network [28] is another reconstruction-based method, in which a neural network is trained to reconstruct input data by minimizing the reconstruction error, and the test sample is classified based on the difference between input and output patterns. Juszczak proposes a novelty detection method based on minimum spanning tree (MST) in [5] to represent the training data in a tree structure, and the distance of the test sample to the tree edge is used as the similarity measure to the target class. Other reconstruction-based methods include the algorithm based on principle component analysis (PCA) [43] or kernel PCA [47] and one-class kernel Fisher's discriminant [48].

Density estimation methods work well when the selected suitable density model has parameters that can be estimated accurately. However, the performance of these methods degrades seriously when the number of training samples is insufficient, particularly for high-dimensional samples. For the boundary and reconstruction methods, their performance may be sensitive to the model parameters, such as the kernel parameter in OCSVM and the number of clusters in k-means clustering.

2.2. One-class support vector machine

Inspired by the binary-class SVM [29,30], Scholkopf et al. propose the OCSVM to solve the one-class classification problem [26]. Consider a training dataset $\{\mathbf{x}_i\}_{i=1}^{N}$, where $\mathbf{x}_i \in \mathbb{R}^{d\times 1}$, d is the original dimension of the training data, and N is the number of training data. Let $\phi(\bullet)$ be a feature mapping, then the kernel function $\kappa(\mathbf{a}_1, \mathbf{a}_2) = \langle \phi(\mathbf{a}_1), \phi(\mathbf{a}_2) \rangle$ corresponds to the inner product in the feature space. The OCSVM tries to construct a hyperplane in the feature space that separates the data from the origin with the maximum margin. In the traditional OCSVM, the typically used transformation method is Gaussian kernel transformation, with which all samples are projected to the hypersphere and move away from the origin in the feature space. The 2D illustration of the Gaussian kernel transformation is shown in Fig. 1.

In Fig. 1, the black solid circles denote the target samples, the blue solid circles denote the non-target samples (or negative samples, outliers), and the red line denotes the optimal hyperplane for classification. Through Gaussian kernel function with the proper Gaussian kernel parameter, all of the samples are separable from the origin, and the target samples and non-target samples are linearly separable in the feature space.

The requirements for the optimal hyperplane in the feature space are as follows: (1) it separates all training data from the origin, and (2) its distance to the origin is maximal. Therefore, the

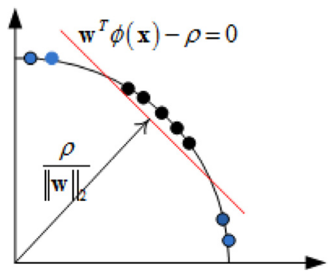


Fig. 1. The 2D illustration of the Gaussian kernel transformation, where the black solid circles denote the positive samples, the blue solid circles denote the negative samples, and the red line denotes the optimal hyperplane for classification.

optimization problem of OCSVM is defined in Eq. (1) as follows:

$$\begin{aligned} &\min_{\mathbf{w},\xi_i,\rho} \tfrac{1}{2}\|\mathbf{w}\|^2 + \tfrac{1}{\eta N}\sum_i \xi_i - \rho \\ &s.t. \quad \left(\mathbf{w}^T\phi(\mathbf{x}_i)\right) \geq \rho - \xi_i \\ &\quad \xi_i \geq 0, \quad \forall i \end{aligned} \tag{1}$$

where $\mathbf{w}$ is the slope of the hyperplane, and ρ is the distance between the origin and the hyperplane, $\mathbf{x}_i$ is the i th sample of the training data, $\phi(\mathbf{x}_i)$ is the projection of $\mathbf{x}_i$ in the feature space, ξ_i is a slack variable corresponding to the i th sample used to model the classification error, and the constant $\eta \in [0,\ 1]$ gives an upper bound on the fraction of outliers and is also varied to obtain the best performance. The decision function of OCSVM can be expressed as follows:

$$f(\mathbf{x}^*) = sign\left(\mathbf{w}^T\phi(\mathbf{x}^*) - \rho\right) \tag{2}$$

Eq. (3) is the dual optimization problem of Eq. (1) with Lagrangian multipliers:

$$\begin{aligned} &\min_{\alpha} \sum_{i,j}^{N} \tfrac{1}{2}\alpha_i\alpha_j\kappa\left(\mathbf{x}_i,\mathbf{x}_j\right) \\ &s.t \quad 0 \leq \alpha_i \leq \tfrac{1}{\eta N}, \forall i \\ &\quad \sum_{i=1}^{N}\alpha_i = 1 \end{aligned} \tag{3}$$

where α_i and α_j are multipliers, and $\kappa(\bullet)$ is the kernel function. The typically used solution to Eq. (3) is sequential minimal optimization (SMO) algorithm [31], which was proposed by Platt in 1998s. Therefore, the decision function with the kernel expansion is formulated as follows:

$$f(\mathbf{x}^*) = sign\left(\sum_{i=1}^{N}\alpha_i\kappa(\mathbf{x}_i,\mathbf{x}^*) - \rho\right) \tag{4}$$

2.3. Dirichlet process mixture

Dirichlet process (DP) [32], first introduced by Ferguson, is a stochastic process applied in nonparametric Bayesian models. The definition of DP is given in

$$G \sim DP(G_0,\alpha) \tag{5}$$

where G is a measure drawn from DP, G_0 is the base distribution, and α is the concentration parameter.

For sampling convenience, Sethuramn proposes a stick-breaking representation for DP [33]. In the stick-breaking representation, the random measure G distributed by a DP with base distribution G_0 and concentration parameter α can be written as follows:

$$\begin{aligned} &G = \sum_{c=1}^{\infty}\pi_c(\mathbf{v})\delta_{\Theta_c} \\ &\pi_c(\mathbf{v}) = v_c\prod_{j=1}^{c-1}\left(1-v_j\right) \\ &v_c \sim Beta(v_c;1,\alpha_0),\ \Theta_c \sim G_0 \end{aligned} \tag{6}$$

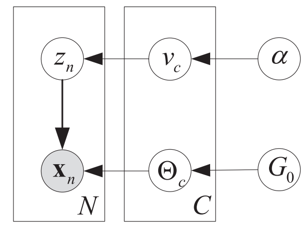


Fig. 2. Graphical model representation of the truncated stick-breaking for DPM.

where $\{\pi_c\}_{c=1}^{\infty}$ are random variables that depend on the parameter α, $\{\Theta_c\}_{c=1}^{\infty}$ are random variables distributed by G_0, and δ_{Θ_c} is an atom at Θ_c.

DP just groups the data with same value into one cluster. Therefore, Antoniak et al. propose the idea of using DP initially as a prior for the mixing proportions of the Gaussian distribution [34]. In the DPM model, the distribution generated from the DP is a prior of the parameter, which parameterizes the distribution of the observed data. The stick-breaking representation of DPM is as follows:

$$\begin{aligned} &v_c \sim Beta(v_c;1,\alpha),\ \{\Theta_c\}_{c=1}^{\infty} \sim G_0 \\ &\pi_c(\mathbf{v}) = v_c\prod_{j=1}^{c-1}\left(1-v_j\right),\ z_i \sim Multi(z_i;\pi) \\ &\mathbf{x}_i|z_i,\{\Theta_c\}_{c=1}^{\infty} \sim F(\mathbf{x}_i;\Theta_{z_i}) \quad i=1,2,\ldots N \\ &G = \sum_{c=1}^{\infty}\pi_c(\mathbf{v})\delta_{\Theta_c} \end{aligned} \tag{7}$$

where $\mathbf{x}_i$ is the training data, z_i is the clustering index of $\mathbf{x}_i$, π_c is the stick weight that represents the probability of the samples belonging to the c th cluster, $Multi(\bullet)$ is the multinomial distribution, and $F(\bullet)$ is the distribution from which the sample $\mathbf{x}_i$ is drawn. The basic idea of DPM is to associate a mixture component with an atom in G. Indicator variables $\{z_i\}_{i=1}^{N}$ are introduced to associate data points with mixture components, and the posterior distribution $F(\bullet)$ yields a probability distribution on partitions of the data [49].

This representation of the DPM clarifies that the support of G consists of a countably infinite set of atoms δ_{Θ_c}, and that the DPM model is a mixture model with a countably infinite number of mixture components. In addition, the DPM model is a nonparametric Bayesian model that can automatically determine the number of clusterings.

Blei et al. introduce the truncated stick-breaking representation for the DPM [35], in which the truncation level C for the number of clusters is fixed and $q(v_C)=1$. The mixture proportion $\pi_c(\mathbf{v})$ is equal to zero for $c>C$, and the clustering number is less than or equal to C. The truncated stick-breaking representation for the DPM can be written as follows:

$$\begin{aligned} &v_c \sim Beta(v_c;1,\alpha),\ \{\Theta_c\}_{c=1}^{C} \sim G_0 \\ &\pi_c(\mathbf{v}) = v_c\prod_{j=1}^{c-1}\left(1-v_j\right),\ z_i \sim Multi(z_i;\pi) \\ &\mathbf{x}_i|z_i,\{\Theta_c\}_{c=1}^{C} \sim F(\mathbf{x}_i;\Theta_{z_i}) \quad i=1,2,\ldots N \\ &G = \sum_{c=1}^{C}\pi_c(\mathbf{v})\delta_{\Theta_c} \end{aligned} \tag{8}$$

We suppose the distribution of c th cluster as $F(\bullet)$ with parameters $\{\Theta_c\}$, and the prior distribution of parameters $\{\Theta_c\}$ as G_0. In the model shown in Eq. (8), $\mathbf{x}_i$ is the input data, and the conditional posterior distribution of the output parameters $\{\Theta_c\}_{c=1}^{C}$, $\{v_c\}_{c=1}^{C}$, z_i can be derived respectively. Markov Chain Monte Carlo (MCMC) approach with a Gibbs sampler is used to estimate these conditional posterior distributions. The truncated stick-breaking representation for the DPM is depicted as a graphical model in Fig. 2. In this study, we use the truncated stick-breaking representation for DMP.

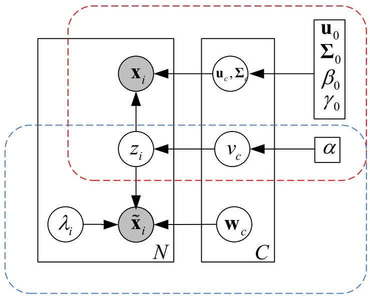


Fig. 3. Graphical model representation of IB-OCSVM.

3. Infinite Bayesian one-class support vector machine

The OCSVM with Gaussian kernel function is the commonly used method for the one-class classification problem. However, the performance of OCSVM is sensitive to the model parameter in the Gaussian kernel transformation. To solve this problem, we propose the IB-OCSVM method. Different from the conventional kernel function which directly optimizes the kernel parameter, IB-OCSVM constructs a linear classification hyperplane in the feature space based on the DPM clustering to realize the complex nonlinear classification. Therefore, IB-OCSVM only requires the linear separability in each cluster in the feature space instead of the linear separability for all transformed features in the feature space in the OCSVM. Our method can reduce sensitivity to the model parameter used in the transformation and enhance classification performance.

Given that we cluster the training data with DPM in the original space and construct a linear hyperplane with the modified OCSVM in each cluster in the feature space, there are two requirements for the transformation: 1) the transformed training samples are separable from the origin, thus guaranteeing the existence of the hyperplane for classification with the same properties of the hyperplane in the traditional OCSVM; and 2) the transformed training samples in the feature space preserve the same clustering characteristic as the original training samples in the original data space, thus guaranteeing the combination of clustering and classification in our method. To satisfy these requirements for transformation, a new transformation method based on the clustering result and Gaussian kernel function is introduced. The traditional OCSVM is modified by replacing the kernel transformation with the new transformation method, which is referred to as the modified OCSVM.

The clustering and classification link up with each other through the joint optimization, which guarantees consistency of the clustering and linear classification in each cluster. To realize such joint optimization, the IB-OCSVM is constructed in the unified Bayesian frame because the DPM model is a nonparametric Bayesian model. Inspired by the idea of [36], we provide a Bayesian solution method for the optimization problem of the modified OCSVM to achieve the joint optimization.

In Section 3.1, we introduce the new transformation method and its optimization problem in the modified OCSVM. Then, the Bayesian solution to the optimization problem of the modified OCSVM is derived in Section 3.2. The complete model of IB-OCSVM is presented in Section 3.3 and the prediction procedure for a new test sample is provided in Section 3.4.

Table 1 lists all notations in our IB-OCSVM model to help readers understand our method.

3.1. New feature transformation in the modified OCSVM

The transformation form is defined as follows:

$$\tilde{\mathbf{x}}_i = [\kappa(\mathbf{x}_i, \mathbf{r}_1), \kappa(\mathbf{x}_i, \mathbf{r}_2), \ldots, \kappa(\mathbf{x}_i, \mathbf{r}_M)] \tag{9}$$

where $\kappa(\mathbf{a}_1, \mathbf{a}_2) = \exp(-||\mathbf{a}_1 - \mathbf{a}_2||_2^2 / 2\sigma^2)$ is the Gaussian kernel function with the kernel parameter σ, $\mathbf{x}_i$ is the original data, $\tilde{\mathbf{x}}_i$ is the projection of $\mathbf{x}_i$ in the feature space, and $\mathbf{r} = \{\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_M\}$ is a set of base vectors selected from the training dataset based on the clustering result via DPM.

The DPM model, introduced in Section 2.2**,** can obtain the underlying structure-distribution of the target data with the number of clusters automatically determined. The base vector set $\mathbf{r} = \{\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_M\}$ is constructed by selecting a fixed proportion η of the training samples from each cluster, Therefore, $\mathbf{r}$ is the representation underlying structure of the training data.

First, the value of $\kappa(\mathbf{x}_i, \mathbf{r}_j)$ is $0 < \kappa(\mathbf{x}_i, \mathbf{r}_j) \le 1$ for any sample $\mathbf{x}_i$ or base vector $\mathbf{r}_j$. Second, due to the difference between the structure-distribution of positive data and that of negative data, for a negative sample $\mathbf{x}_i$, the value of $\kappa(\mathbf{x}_i, \mathbf{r}_j)$ is close to zero for any base vector $\mathbf{r}_j$; while due to the similarity of the positive samples belonging to the same cluster, for a positive data $\mathbf{x}_i$, the value of $\kappa(\mathbf{x}_i, \mathbf{r}_j)$ is close to one of the base vector $\mathbf{r}_j$ belonging to the same cluster of $\mathbf{x}_i$. Therefore, all transformed samples lie within the hypersquare in the first quadrant, with the negative samples scattering around the origin while the positive samples scatter around each vertex of the hyperplane except for the origin. All transformed target samples also preserve the same clustering characteristic as the original samples in the original space because the original target samples belonging to the same cluster are projected in the close positions in the feature space.

When kernel transformation is replaced with the new feature transformation, the optimization problem of Eq. (1) for OCSVM is expressed as follows:

$$\begin{aligned} &\min_{\mathbf{w}, \xi_i, \rho} \quad \tfrac{1}{2}\|\mathbf{w}\|^2 + \tfrac{1}{\eta N}\sum_i \xi_i - \rho \\ &s.t. \quad \left(\mathbf{w}^T \tilde{\mathbf{x}}_i\right) \ge \rho - \xi_i \\ &\quad\ \xi_i \ge 0, \quad \forall i \end{aligned} \tag{10}$$

Compared with Eq. (1), the projected sample $\phi(\mathbf{x}_i)$ is replaced with the new transformed data $\tilde{\mathbf{x}}_i$, and $\tilde{\mathbf{x}}_i$ has an explicated expression while $\phi(\mathbf{x}_i)$ does not.

In the traditional OCSVM introduced in Section 2.1, the classification hyperplane is $f(\mathbf{x}) = \mathbf{w}^T\phi(\mathbf{x}) - \rho = 0$ with the parameters $\mathbf{w}$ and ρ. If we set the parameter ρ as a nontrivial constant, e.g., $\rho = 1$, then the classification hyperplane becomes $\hat{f}(\mathbf{x}) = \mathbf{w}^T\phi(\mathbf{x}) - 1 = 0$ with the parameter $\mathbf{w}$. The only difference between hyperplanes $f(\mathbf{x})$ and $\hat{f}(\mathbf{x})$ is that hyperplane $f(\mathbf{x})$ can pass through the origin for $\rho = 0$. In the OCSVM, we construct the hyperplane to separate the training data from the origin with a maximum margin, thus the hyperplane constructed in the OCSVM will definitely not pass through the origin, which allows us to use the hyperplane $\hat{f}(\mathbf{x}) = \mathbf{w}^T\phi(\mathbf{x}) - 1 = 0$ to replace the hyperplane $f(\mathbf{x}) = \mathbf{w}^T\phi(\mathbf{x}) - \rho = 0$. Therefore, the optimization problem of Eq. (10) for OCSVM can be rewritten as follows:

$$\begin{aligned} &\min_{\mathbf{w}, \xi_i} \quad \tfrac{1}{2}\|\mathbf{w}\|^2 + \tfrac{1}{\eta N}\sum_i \xi_i - 1 \\ &s.t. \quad \left(\mathbf{w}^T \tilde{\mathbf{x}}_i\right) \ge 1 - \xi_i, \forall i \\ &\quad\ \xi_i \ge 0, \forall i \end{aligned} \tag{11}$$

Eq. (11) can be transformed to an unconstrained optimization problem as

$$\min_{\mathbf{w}} d(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2 + 2l\sum_i \max\left(1 - \mathbf{w}^T\tilde{\mathbf{x}}_i, 0\right) \tag{12}$$

where $2l = 1/(\eta N)$.

Table 1
List of Notations.

Distribution	$Beta(\bullet)$	Beta distribution
	$NW(\bullet)$	Normal-Wishart distribution
	$Multi(\bullet)$	Multinomial distribution
	$N(\bullet)$	Normal distribution
Function	$\exp(\bullet)$	Exponential function
Variable	v_c	Random variable that controls the mixing proportions
	π_c	Mixing proportion
	z_i	Cluster index of sample $\mathbf{x}_i$
	$\{\mathbf{u}_c, \mathbf{\Sigma}_c\}$	Mean vector and covariance matrix of Normal distribution in the c th cluster
	$\mathbf{w}_c$	Slope of classification hyperplane in the c th cluster
	λ_i	Latent variable
Hyperparameters	α	Hyperparameter of beta distribution
	$\mathbf{u}_0, \mathbf{\Sigma}_0$ β_0, γ_0	Hyperparameter of Normal-Wishart distribution
Other Notations	l	Penalty parameter in the modified OCSVM
	$\mathbf{x}_i$	Original training sample
	$\tilde{\mathbf{x}}_i$	Training sample in the feature space

Algorithm 1 Procedure for feature transformation.

Input: Training dataset $\mathbf{x} = \{\mathbf{x}_i\}_{i=1}^N$, clustering index $z = \{z_i\}_{i=1}^N$, kernel parameter σ, truncated level C, and parameter η;
Output: Transformed dataset $\tilde{\mathbf{x}} = \{\tilde{\mathbf{x}}_i\}_{i=1}^N$, base vector set $\mathbf{r} = \{\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_M\}$;
1: **for** $c = 1: C$
2: Randomly select samples belonging to the c th cluster from the training dataset $\mathbf{x}$ with a fixed ratio η based on the clustering index $\mathbf{z} = \{z_i\}_{i=1}^N$;
3: Put the selected samples into the base vector set $\mathbf{r}$;
4: **end for**
5: **for** $i = 1: N$
6: Transform the training sample $\mathbf{x}_i$ to the feature space with the formulation $\tilde{\mathbf{x}}_i = [\kappa(\mathbf{x}_i, \mathbf{r}_1), \kappa(\mathbf{x}_i, \mathbf{r}_2), \ldots, \kappa(\mathbf{x}_i, \mathbf{r}_M)]$, where $\kappa(\bullet)$ is the Gaussian kernel function with kernel parameter σ;
7: **end for**

3.2. Bayesian solution for the optimization problem of the modified OCSVM

The Bayesian solution for optimization problem shown in Eq. (12) is now provided to help realize the joint optimization of clustering and classification in our IB-OCSVM.

With the idea of [36], the optimal parameter value of $\mathbf{w}$ can be obtained as follows:

$$\mathbf{w}_{opt} = \int p(\mathbf{w}|\tilde{\mathbf{x}})d\mathbf{w} \tag{13}$$

where $\tilde{\mathbf{x}} = \{\tilde{\mathbf{x}}_i\}_{i=1}^N$ is the transformed training dataset in the feature space, the pseudo-posterior distribution $p(\mathbf{w}|\tilde{\mathbf{x}})$ is defined by:

$$p(\mathbf{w}|\tilde{\mathbf{x}}) \propto \exp\left(-d(\mathbf{w})\right) \propto p(\mathbf{w})p\left(\hat{\mathbf{x}}|\mathbf{w}\right) \tag{14}$$

The data-independent term in Eq. (14) can be regarded as a Gaussian prior of the parameter $\mathbf{w}$:

$$p(\mathbf{w}) = \exp\left(-\frac{1}{2}\mathbf{w}^T\mathbf{w}\right) \tag{15}$$

The data-dependent term in Eq. (14) can be regarded as a pseudo-likelihood:

$$p(\tilde{\mathbf{x}}|\mathbf{w}) = \prod_i p_i(\tilde{\mathbf{x}}_i|\mathbf{w}) = \exp\left(-2l\sum_i \max\left(1 - \mathbf{w}^T\tilde{\mathbf{x}}_i, 0\right)\right) \tag{16}$$

Eq. (16) can be rewritten as a conditional Gaussian linear model with latent variable λ_i by using the data augmentation [37]:

$$\begin{aligned} p(\tilde{\mathbf{x}}_i|\mathbf{w}) &= \exp(-2l\max(1 - \mathbf{w}^T\tilde{\mathbf{x}}_i, 0)) \\ &= \int_0^\infty \frac{1}{\sqrt{2\pi\lambda_i}} \exp\left(-\frac{(l - l\mathbf{w}^T\tilde{\mathbf{x}}_i + \lambda_i)^2}{2\lambda_i}\right)d\lambda_i \end{aligned} \tag{17}$$

The operation of $\max(\bullet)$ in Eq. (16) is replaced by the integration in Eq. (17) with the latent variable λ_i , which is convenient for conditional posterior inference. With Eq. (17) Substituted into Eq. (14), the pseudo-posterior distribution $p(\mathbf{w}|\tilde{\mathbf{x}})$ can be expressed as follows:

$$p(\mathbf{w}|\tilde{\mathbf{x}}) = \int p(\mathbf{w}, \lambda|\tilde{\mathbf{x}})d\lambda \tag{18}$$

where the complete data pseudo-posterior distribution is as follows:

$$\begin{aligned} &p(\mathbf{w}, \lambda|\tilde{\mathbf{x}}) \propto p(\tilde{\mathbf{x}}, \lambda|\mathbf{w})p(\mathbf{w}) \\ &\propto \prod_i \frac{1}{\sqrt{2\pi\lambda_i}} \exp\left\{-\frac{(l - l\mathbf{w}^T\tilde{\mathbf{x}}_i + \lambda_i)^2}{2\lambda_i}\right\} \times \exp\left(-\tfrac{1}{2}\mathbf{w}^T\mathbf{w}\right) \end{aligned} \tag{19}$$

According to Eqs. (12) and (18), the optimal parameter value of $\mathbf{w}$ can be obtained as follows:

$$\mathbf{w}_{opt} = \int\int p(\mathbf{w}, \lambda|\tilde{\mathbf{x}})d\lambda d\mathbf{w} \tag{20}$$

Markov chain Monte Carlo (MCMC) algorithm [38,39] is used to solve the optimization problem of Eq. (20). The conditional distributions $p(\mathbf{w}|\tilde{\mathbf{x}}, \lambda)$ and $p(\lambda|\tilde{\mathbf{x}}, \mathbf{w})$ can be inferred with Eq. (19), and then the Gibbs sampling algorithm is used to estimate the parameter $\mathbf{w}_{opt}$. In the Gibbs sampler, there are two steps: burn-in step and collection step. In the burn-in step, the parameters are iteratively updated for a certain number of iterations to ensure the convergence of the algorithm. When the algorithm has converged, samples are collected to estimate the parameters, which is termed as the collection step. In our paper, $I_{burn-in}$ is the number of iterations in burn-in step, and I is the sum of the number of iterations in the burn-in and collection steps. The procedure of Gibbs sampling is given below.

1. Initialize parameters $\mathbf{w}$ and λ;
2. **For** $\tau=1:I$
3. Sample $\mathbf{w}^{(\tau+1)} \sim p(\mathbf{w}|\tilde{\mathbf{x}}, \lambda^{(\tau)})$;
4. Sample $\lambda^{(\tau+1)} \sim p(\lambda|\tilde{\mathbf{x}}, \mathbf{w}^{(\tau+1)})$;
5. **If** $\tau > I_{burn-in}$
6. Collect parameters $\mathbf{w}$ and λ;
7. **End if**
8. **End for**

3.3. Infinite Bayesian one-class support vector machine

In Section 2.3, we introduced the DPM clustering method. In Sections 3.1 and 3.2, we developed the modified OCSVM with the new feature transformation under the Bayesian framework. In the following, the clustering is combined with the classification through the joint Bayesian optimization, and the unified Bayesian model is referred to as IB-OCSVM. In the clustering step, we use the DPM to cluster the training data; in the classification step, we construct a linear hyperplane in each cluster in the feature space with the modified OCSVM. The clustering and classification steps link up with each other through joint optimization to mutually reinforce each other and guarantee the consistency of clustering and linear classification in each cluster in the feature space. Since Gaussian mixture model (GMM) is commonly used as a parametric model of the probability distribution of continuous measurements or features [50], we suppose the distribution of features is GMM in the DPM model i.e., the distribution of each cluster is Gaussian. Then we set the base distribution as Normal-Wishart distribution, the conjugate distribution of Gaussian distribution, which is convenient for us to derive the posterior distribution. Our IB-OCSVM model is shown in Eqs. (21) and (22):

$$\text{Clustering} \quad \begin{cases} v_c \sim Beta(v_c; 1, \alpha), \{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C} \\ \sim NW(\mathbf{u}_c, \boldsymbol{\Sigma}_c; \mathbf{u}_0, \boldsymbol{\Sigma}_0, \beta_0, \gamma_0) \\ \pi_c(\mathbf{v}) = v_c \prod_{j=1}^{c-1} \left(1 - v_j\right), z_i \sim Multi(z_i; \pi) \\ \mathbf{x}_i | z_i, \{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{\infty} \sim N(\mathbf{x}_i; \mathbf{u}_{z_i}, \boldsymbol{\Sigma}_{z_i}) \end{cases} \tag{21}$$

$$\text{Classification} \quad \begin{cases} \{\mathbf{w}_c\}_{c=1}^{C} \sim N(\mathbf{w}_c; 0, \mathbf{I}) \\ \tilde{\mathbf{x}}_i, \lambda_i | z_i, \{\mathbf{w}_c\}_{c=1}^{C} \propto \frac{1}{\sqrt{2\pi\lambda_i}} \exp\left\{ -\frac{(l - l\mathbf{w}_{z_i}{}^T\tilde{\mathbf{x}}_i + \lambda_i)^2}{2\lambda_i} \right\} \end{cases} \tag{22}$$

where $\mathbf{w}_c$ is the slope of the hyperplane in the c th cluster with $c = 1, 2, \ldots, C$, $NW(\bullet)$ is the Normal–Wishart distribution. Eq. (21) defines the optimization problem of the DPM model, and Eq. (22) defines the optimization problem of the modified OCSVM model for each cluster. Eq. (21) is a specific form of Eq. (8), which is the truncated stick-breaking representation of DPM. The difference between Eq. (8) and Eq. (21) is that we suppose the distribution $F(\bullet)$ as Normal distribution and the base distribution G_0 as a Normal–Wishart distribution in Eq. (21). In the IB-OCSVM, we learn a modified OCSVM in each cluster. Therefore, we need to learn a set of parameters $\{\mathbf{w}_c\}_{c=1}^{C}$ and each parameter $\mathbf{w}_c$ is estimated only using the samples belonging to c th cluster based on the clustering index z_i in Eq. (22).

Clearly, the clustering is conducted in the original space, whereas the hyperplane construction is conducted in the feature space. The clustering and the classification steps can be combined in our method, because the transformed training data in the feature space, $\tilde{\mathbf{x}} = \{\tilde{\mathbf{x}}_i\}_{i=1}^{N}$, can preserve the same clustering characteristic as the original training samples in the original data space, $\mathbf{x} = \{\mathbf{x}_i\}_{i=1}^{N}$. The combination is realized by sharing the clustering index z_i between the clustering and classification steps.

Through Eqs. (21) and (22), the complete pseudo-posterior distribution can be obtained as follows:

$$\begin{aligned} & p\left(\{\mathbf{w}_c\}_{c=1}^{C}, \{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}, \mathbf{v}, \lambda, \mathbf{z} | \mathbf{x}, \tilde{\mathbf{x}}\right) \\ & \propto p\left(\{\mathbf{w}_c\}_{c=1}^{C}, \{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}, \mathbf{v}, \lambda, \mathbf{z}, \mathbf{x}, \tilde{\mathbf{x}}\right) \\ & \quad = p\left(\mathbf{x} | \{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}, \mathbf{z}\right) \prod_{c=1}^{C} p(\mathbf{u}_c, \boldsymbol{\Sigma}_c | \mathbf{u}_0, \boldsymbol{\Sigma}_0, \beta_0, \gamma_0) \\ & \quad \cdot p(\mathbf{z}|\mathbf{v}) p(\mathbf{v}|\alpha_0) \prod_{c=1}^{C} p(\tilde{\mathbf{x}}, \lambda | \mathbf{w}_c) p(\mathbf{w}_c) \end{aligned} \tag{23}$$

where $\mathbf{z} = \{z_i\}_{i=1}^{N}$, $\lambda = \{\lambda_i\}_{i=1}^{N}$. The conditional posteriors used to draw samples can be derived via Eq. (23). The posterior computation with the MCMC method based on Gibbs sampling is realized in the present study [38,39].

- Sample the Gaussian distribution parameter $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}$ of each cluster from $p(\mathbf{u}_c, \boldsymbol{\Sigma}_c|-) \sim NW(\mathbf{u}_c, \boldsymbol{\Sigma}_c; \mathbf{u}, \boldsymbol{\Sigma}, \beta, \gamma)$ with

$$\begin{aligned} \beta &= \beta_0 + N_c \\ \gamma &= \gamma_0 + N_c \\ \mathbf{u} &= (N_c\tilde{\mathbf{m}}_c + \beta_0\boldsymbol{\mu}_0)/\beta \\ \boldsymbol{\Sigma} &= \left[\boldsymbol{\Sigma}_0^{-1} + N_c\tilde{\mathbf{S}}_c + N_c\beta_0(\tilde{\mathbf{m}}_c - \boldsymbol{\mu}_0)(\tilde{\mathbf{m}}_c - \boldsymbol{\mu}_0)^T/\beta\right]^{-1} \end{aligned} \tag{24}$$

  where N_c is the number of samples belonging to the c th cluster, $\tilde{\mathbf{m}}$ is the mean of maximum likelihood estimation for the c th cluster, and $\tilde{\mathbf{S}}_c$ is the covariance matrix of maximum likelihood estimation for the c th cluster.
- Sample the clustering index z_i corresponding to the sample $\mathbf{x}_i$ from $p(z_i|-) \sim Multi(z_i; [T_{i1}, T_{i2}, \ldots T_{iC}])$ with

$$\begin{aligned} & T_{ic} \triangleq p(z_i = \mathrm{c}|-) \propto \pi_c \cdot N\left(\mathbf{x}_i; \mu_c, \boldsymbol{\Sigma}_c^{-1}\right) \\ & \quad \times \exp\left(-\frac{(l - l\mathbf{w}_c^T\tilde{\mathbf{x}}_i + \lambda_i)^2}{2\lambda_i}\right) \end{aligned} \tag{25}$$

- Sample the parameter v_c from $p(v_c|-) \sim Beta(v_c; a, b)$ with

$$\begin{aligned} a &= 1 + N_c \\ b &= \alpha + \sum_{j=c+1}^{C} N_j \end{aligned} \tag{26}$$

- Sample the parameter $\mathbf{w}_c$ from $p(\mathbf{w}_c|-) \sim N(\mathbf{w}_c; \mathbf{E}_c, \boldsymbol{\Lambda}_c)$ with

$$\begin{aligned} \boldsymbol{\Lambda}_c &= \left(\mathbf{I}_M + l^2 \sum_{i=1}^{N} \frac{\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^T}{\lambda_i} \cdot I(z_i = c)\right)^{-1} \\ \mathbf{E}_c &= l \cdot \boldsymbol{\Lambda}_c \cdot \sum_{i=1}^{N} \left[\left(1 + \frac{l}{\lambda_i}\right)\tilde{\mathbf{x}}_i \cdot I(z_i = c)\right] \end{aligned} \tag{27}$$

- Sample the latent variable parameter λ_i^{-1} from $p(\lambda_i^{-1}|-) \sim IG(\lambda_i^{-1}; \delta, \eta)$ with

$$\begin{aligned} \delta &= \left|l - l\mathbf{w}_{z_i}^T\tilde{\mathbf{x}}_i\right|^{-1} \\ \eta &= 1 \end{aligned} \tag{28}$$

  where $IG(\bullet)$ is the inverse Gaussian distribution.

On the basis of conditional distributions given above and Gibbs sampling, we can construct the Markov chain to draw samples iteratively with an initial condition (Algorithm 2).

3.4. Prediction procedure for a new sample

For a new sample $\mathbf{x}^*$, the classification result is given by the following steps:

Algorithm 2 Training step of the IB-OCSVM method.

Input: $\mathbf{x}=\{\mathbf{x}_i\}_{i=1}^{N}$, I, $I_{burn-in}$, $\mathbf{u}_0$, Σ_0, α, β_0, γ_0, threshold ε_{thres};
Output: Parameters $\{\mathbf{w}_c\}_{c=1}^{C}$, $\mathbf{v}$, $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}$;
Initialize the parameter $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}$, $\mathbf{z}$, $\boldsymbol{\lambda}$, $\mathbf{v}$, $\{\mathbf{w}_c\}_{c=1}^{C}$.
for $t=1$ to I **do**
  Sample the parameter $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}$ from Eq. (24);
  Sample the parameter $\mathbf{z}$ from Eq. (25);
  Sample the parameter $\mathbf{v}$ from Eq. (26);
  Apply Algorithm 1 to transform the training samples to feature space;
  Sample the parameters $\{\mathbf{w}_c\}_{c=1}^{C}$ from Eq. (27);
  Sample the parameters $\boldsymbol{\lambda}$ from Eq. (28);
  Computing the relative of change of classification accuracies between adjacent samples ε;
if $t > I_{burn-in}$ **or** $\varepsilon < \varepsilon_{thres}$, **then**
  Collect parameters $\{\mathbf{w}_c\}_{c=1}^{C}$, $\mathbf{v}$, $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}_{c=1}^{C}$;
end if
end for

Table 2
Computational complexity analysis of IB-OCSVM at each iteration.

Parameter	$\{u_c, \Sigma_c\}$	z	v	w_c	λ
computational complexity	$o(d^3)$	$o(NC)$	$o(C^2)$	$o(NM^2+M^3)$	$o(NM)$

- Computing the clustering index z_t^* based on

$$z_t^* \sim Multi\left(T_{1,t}^*, T_{2,t}^*, \ldots, T_{C,t}^*\right)$$
$$T_{c,t}^* = p(z_t^* = c|-) \propto \pi_c^t(\mathbf{v}_t) \cdot N\left(\mathbf{x}^*; \mathbf{u}_c^t, \boldsymbol{\Sigma}_c^t\right) \tag{29}$$

where $\{\mathbf{u}_c^t, \boldsymbol{\Sigma}_c^t\}$ is the mean vector and covariance matrix of the c th cluster collected in the t th time. $\mathbf{v}_t$ represents the parameter $\mathbf{v}$ collected in the t th time.
- Transforming the test sample $\mathbf{x}^*$ from the original space to the feature space:

$$\tilde{\mathbf{x}}_t^* = \left[\kappa\left(\mathbf{x}^*, \mathbf{r}_1^t\right), \kappa\left(\mathbf{x}^*, \mathbf{r}_2^t\right), \ldots, \kappa\left(\mathbf{x}^*, \mathbf{r}_M^t\right)\right] \tag{30}$$

where $\tilde{\mathbf{x}}_t^*$ is the transformed sample based on the base vector set $\mathbf{r}^t = [\mathbf{r}_1^t, \mathbf{r}_2^t, \ldots \mathbf{r}_M^t]$ collected in the t th time.
- Computing the final classification label y^* based on

$$y^* = sign\left(\frac{1}{T - T_{burn-in}} \sum_{t=T_{burn-in}+1}^{T} (< \mathbf{w}_{z_t^*}^t, \tilde{\mathbf{x}}_t^* > -1)\right) \tag{31}$$

where $\mathbf{w}_{z_t^*}^t$ represents the slope of the hyperplane in the z_t^* th cluster sampled in the t th time.

3.5. Computational complexity analysis

In this Section, *computational complexity analysis* of IB-OCSVM is given. In Table 2, we list the complexity of each parameter at each iteration in our method. where N is the number of training data, d is the original dimension of the data, C is the truncation level, and M is the number of base vectors used in the feature transformation. For our method, to sample the parameter $\{\mathbf{u}_c, \boldsymbol{\Sigma}_c\}$, we need to inverse a $d \times d$ matrix, which is time-consuming for high-dimension data. For other parameters, it is not time-consuming to sample them since $M \ll d$.

4. Experiments and results

First, we give an explanation about the experiment condition and the parameter settings. In this paper, all algorithms for our experiments are conducted using a PC with Intel(R) Core i5-4590 CPU of 3.3 GHz and memory of 8 GB. The program codes are written in MATLAB. The parameter settings are as follows:

In general, the fixed number of epochs (i.e. w.r.t. $I_{burn-in}$) for the Gibbs sampling procedure as a termination criteria is data and application dependent. In our experiments, in order to ensure the convergence of the algorithm for all datasets used in the paper, the number of burn-in samples is set as $I_{burn-in} = 3000$. The number of collection samples is set as $I_{num} = 300$, and the sampling interval I_{space} is set as 10 to eliminate the interrelation of adjacent collection samples. The truncation level C is set to be 10 because the truncation level C can be set as any value lager than the real number of clusters of the dataset. The values of hyperparameters used in the Normal–Wishart distribution $NW(\mathbf{u}_0, \boldsymbol{\Sigma}_0, \beta_0, \gamma_0)$ are set to be $NW(\mathbf{u}_\mathbf{x}, \boldsymbol{\Sigma}_\mathbf{x}, 1, d)$, which is same as the corresponding prior used in [40]. $\mathbf{u}_\mathbf{x}$ and $\boldsymbol{\Sigma}_\mathbf{x}$ are the mean vector and covariance matrix of training data, and d is original dimension of training sample. In the hierarchical Bayesian model, the values of hyperparameters have little effect on final performance [41]. Therefore, hyperparameter α is set as middle value $\alpha = 0.5$ because of lacking the prior information about this hyperparameter.

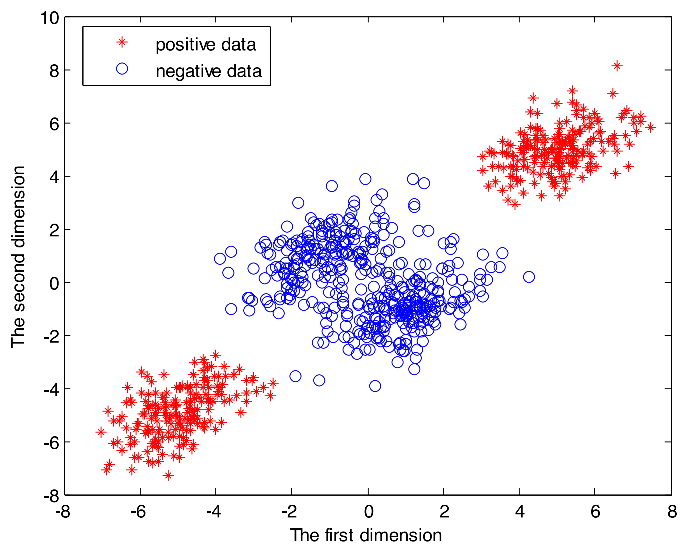


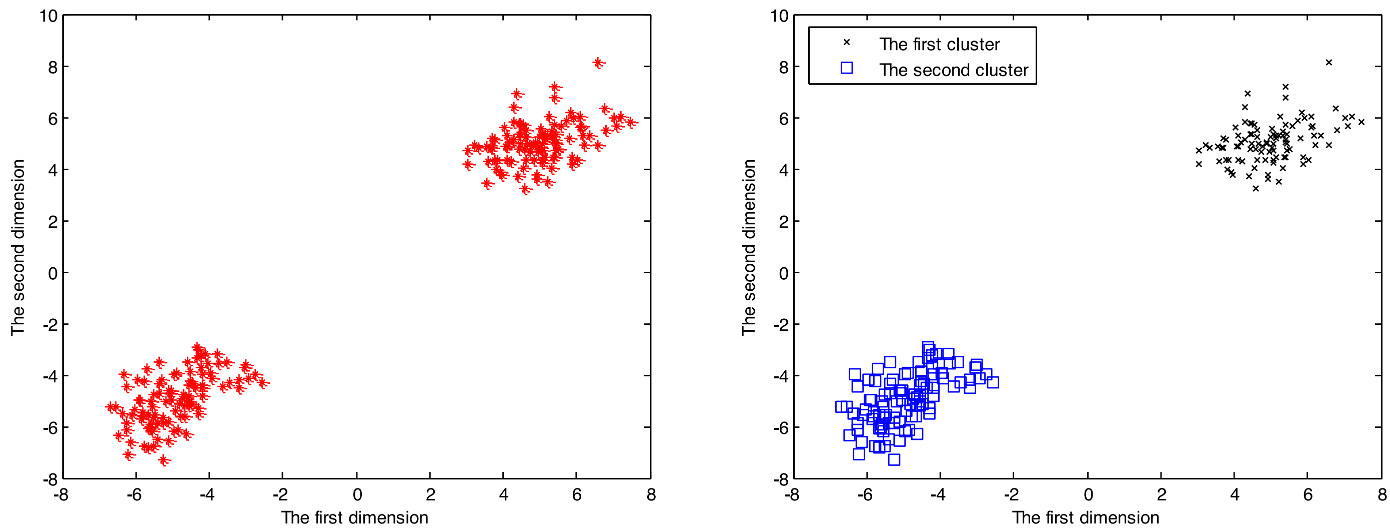
Fig. 4. All original samples in the 2D plane.

4.1. Visual results on synthetic dataset

800 samples generated from a mixture of 4 Gaussian distributions are used in this experiment. The samples drawn from two of the Gaussian distributions are set as positive samples, and the remaining samples are set as negative samples. The distribution of original samples in the 2D plane is shown in Fig. 4. In this experiment, the kernel parameter σ is set to be 2, and parameter η is set to be 0.2.

Table 3
Description of the benchmark datasets.

Dataset	Number of classes	The positive class	Data dimension	Number of training data	Number of the test positive data	Number of the test negative data
Australian	2	1	14	215	92	92
Breast_cancer	2	1	90	116	80	81
Landsat	6	4,5,6	36	500	418	1082
Pendigits	10	10	16	572	571	1000
Waveform	3	1,2	21	331	1652	1696
Cleve	2	1	13	112	48	48



(a) Training data in the in the 2D plane; (b) Clustering result via DPM

Fig. 5. The original distribution and the clustering result of the training data.

Table 4
Average values (%) and standard deviations (%) of three criterions for Waveform data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	57.05 ± 2.10	67.12 ± 1.11	71.19 ± 2.81
KOCSVM	64.14 ± 1.48	59.94 ± 2.75	69.63 ± 2.16
k-means	66.94 ± 1.18	71.92 ± 0.88	76.71 ± 3.61
PCA	55.26 ± 0.69	66.18 ± 0.33	59.50 ± 1.98
Gauss	65.09 ± 2.02	71.60 ± 0.94	80.55 ± 1.85
MST	61.19 ± 1.63	70.70 ± 1.56	73.57 ± 1.56
SOM	67.74 ± 1.18	68.30 1.52	73.83 ± 1.40
Auto-encoder	58.00 ± 2.22	56.40 ± 3.79	61.26 ± 2.90
MPM	72.44 ± 1.24	65.66 ± 4.52	***83.98 ± 1.48***
LPDD	74.25 ± 2.15	71.02 ± 3.07	82.01 ± 2.29
IB-OCSVM	***75.92 ± 1.50***	***74.12 ± 2.30***	83.67 ± 1.50

Table 5
Average values (%) and standard deviations (%) of three criterions for Pendigits data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	89.38 ± 1.16	85.97 ± 1.27	93.87 ± 0.96
KOCSVM	95.46 ± 0.56	93.76 ± 0.72	98.09 ± 0.24
k-means	96.58 ± 0.56	95.06 ± 0.84	99.15 ± 0.54
PCA	96.67 ± 0.55	95.20 ± 0.82	99.26 ± 0.49
Gauss	96.51 ± 0.70	94.95 ± 1.06	99.43 ± 0.39
MST	97.15 ± 0.58	95 91 ± 0.87	***99.89 ± 0.12***
SOM	94.94 ± 0.89	92.50 ± 1.40	99.84 ± 0.12
Auto-encoder	94.32 ± 2.31	91.85 ± 3.01	98.80 ± 1.84
MPM	96.33 ± 0.60	94.68 ± 0.91	99.48 ± 0.37
LPDD	95.90 ± 0.70	94.02 ± 1.07	99.02 ± 0.41
IB-OCSVM	***98.24 ± 0.33***	***97.53 ± 0.48***	99.67 ± 0.17

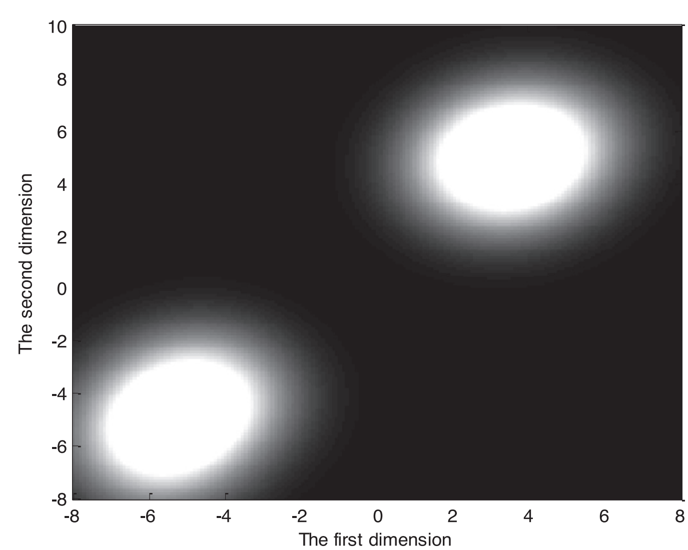
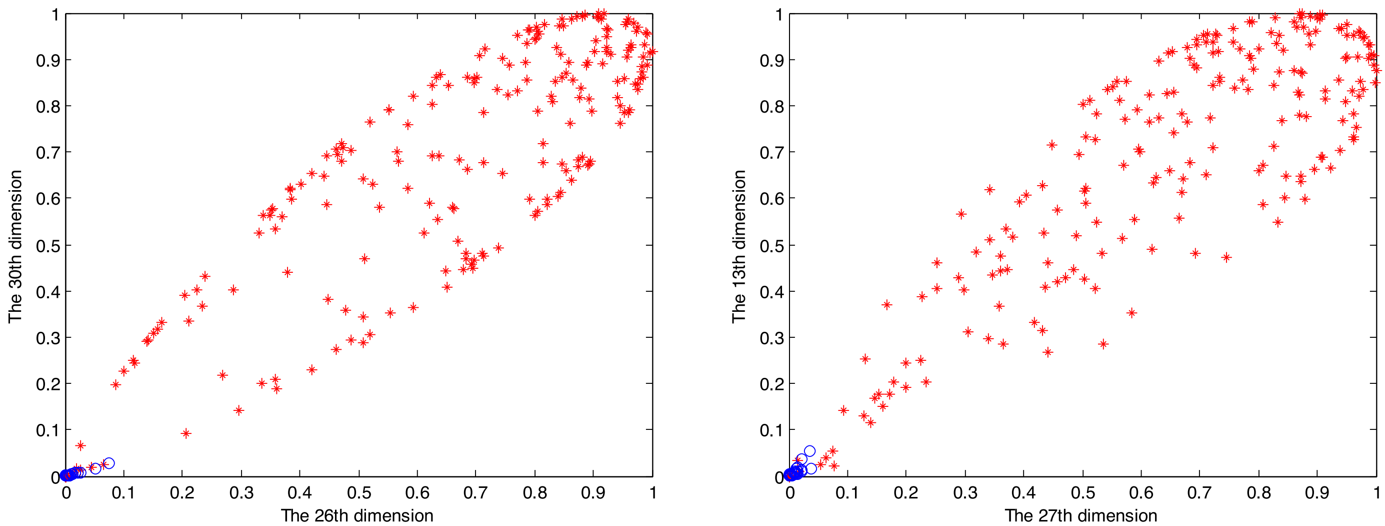


Fig. 6. Classification results in the 2D original space.

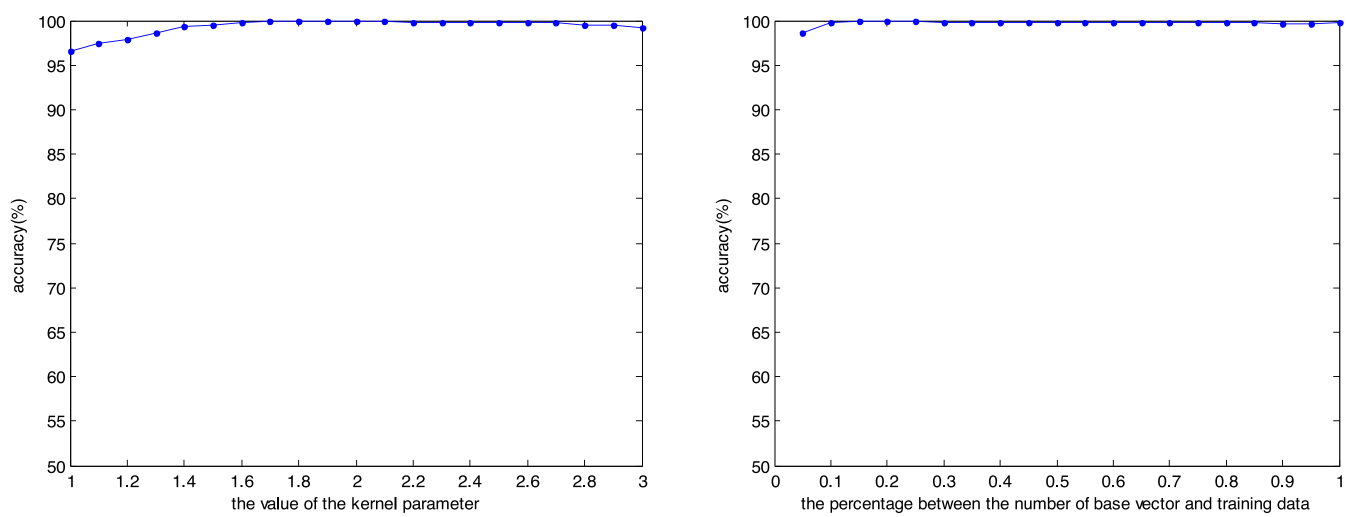
Training dataset is constructed by selecting 100 positive samples from each Gaussian distribution, and test dataset is constructed by the remaining samples are used as the test samples. Fig. 5(a) and (b) show the distribution of training samples and the clustering result of the training samples via DPM clustering. Apparently, the clustering characteristic of the data is learned well via the DPM model.



(a) 26 *th* versus 3 *th* ;

(b) 27 *th* versus 13 *th*

Fig. 7. Feature distribution in the 2-dimensional plane for the transformed features.



(a) Classification accuracies with different values of σ ;

(b) Classification accuracies with different values of η

Fig. 8. Classification accuracies of IB-OCSVM with different values of model parameters.

The classification score of all points in the 2D original space is depicted in Fig. 6, in which the white color represents the value of 1 and the black color represents the value of 0. In Fig. 6, the positions of positive samples are brightest while the other positions are dark, which illustrates the effectiveness of the IB-OCSVM.

The 2D feature distribution of the transformed features is presented in Fig. 7. The top four scored features are chosen via Linear Discriminant Analysis (LDA) [42]. As shown in Fig. 7, the negative samples are close to the origin in the transformed space using our transformation method, thus our transformation method is feasible.

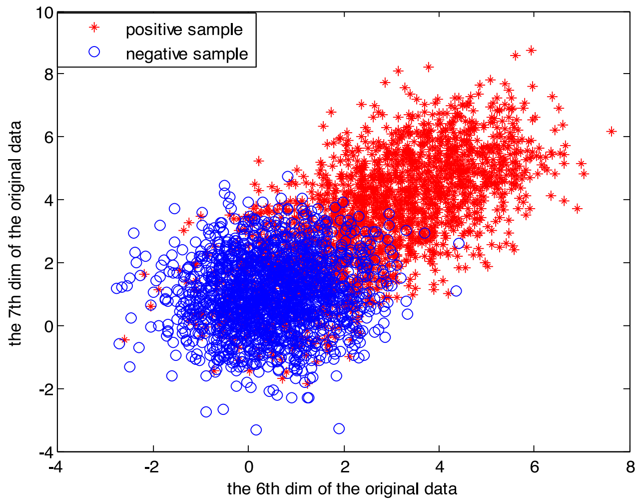
In the experiments above, we set the model parameters $\sigma = 2$ and $\eta = 0.2$. In order to evaluate the robustness of IB-OCSVM, we report the classification performance on the test dataset by varying the values of the model parameters respectively. Fig. 8(a) shows classification accuracies with the different value of kernel parameter, where the value of kernel parameter σ varies from 1 to 3 with the interval of 0.1 and η is set to be 0.2; Fig. 8(b) presents classification accuracies with the different value of η, where the value of model parameter η varies from 0.05 to 1 with the interval of 0.05 and σ is set to be 2. We can see from Fig. 8 that classification accuracies are stable when the model parameters vary.

4.2. Results on benchmark datasets

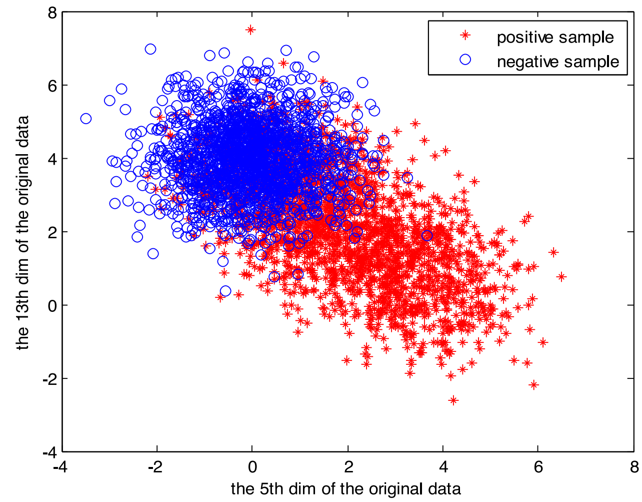
In Section 4.1, we evaluate classification performance of IB-OCSVM on the synthetic dataset. Now, the classification performance of IB-OCSVM on the benchmark datasets is given as follows. A brief introduction about the benchmark datasets used in the experiments is shown in Table 3.[1]

Ten related methods are considered to compare with the proposed method: Gaussian density estimation (Gauss) [28], PCA

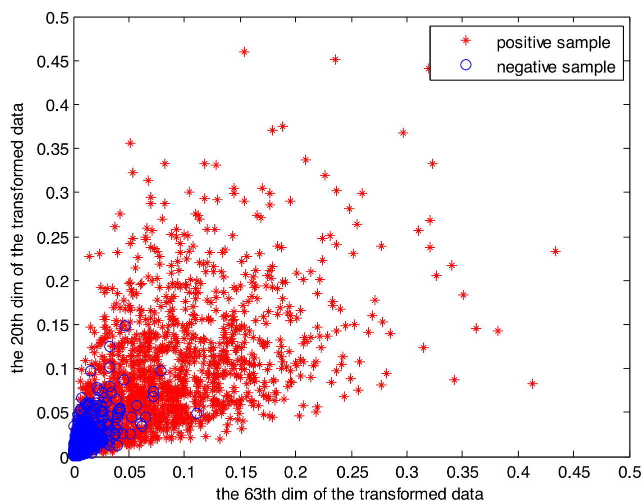
[1] The datasets *Australian, Breast_cancer, Landsat, Pendigits, Waveform and Cleve* can be downloaded from the UCI Machine Learning Repository (http://archive.ics.uci.edu/ml/datasets.html).



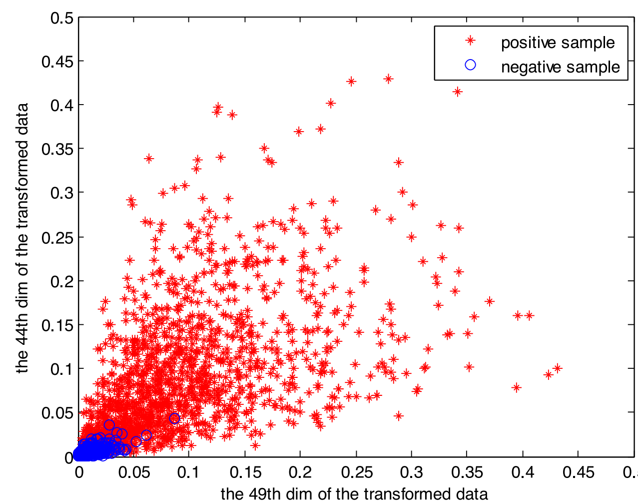
(a) 6 *th* versus 7 *th* for the original features;



(b) 5 *th* versus 13 *th* for the original features



(c) 63 *th* versus 20 *th* for the transformed features;



(d) 49 *th* versus 44 *th* for the transformed features

Fig 9. The 2D feature distribution of original data and transformed data on Waveform dataset.

[43], *k*-means clustering (*k*-means) [28], linear SVDD (L-SVDD) [4], OCSVM with Gaussian kernel function (KOCSVM) [26], MST [5], Self-Organizing Map (SOM) [44], Auto-encoder neural network [28], MPM [27], and LPDD [9]. Because the SVDD with Gaussian kernel function is equal to the KOCSVM, we use the SVDD without kernel transformation, i.e., L-SVDD.

For IB-OCSVM, $\eta = 0.2$, and the value of the kernel parameter σ varies from 2 to 4 at intervals of 0.1. For KOCSVM, the value of the kernel parameter varies from 1 to 10 at intervals of 0.1. For *k*-means, the number of clusters varies from 1 to *C* at intervals of 1. For PCA , the fraction parameter varies from 0.1 to 1 at intervals of 0.1. For Gauss, the regularized parameter varies from 0 to 1 at intervals of 0.1. For SOM, the size of the map varies from 5 to 10 at intervals of 1. For Auto-encoder, the value of hidden units is selected from the set {16, 32, 64, 128, 256, 512, 1024}. For MPM, the value of the kernel parameter varies from 1 to 10 at intervals of 0.1. For LPDD, the value of the parameter used in the sigmoidal transformation varies from 1 to 10 at intervals of 0.1.The model parameters are selected corresponding to the best performance for all methods.

We evaluate the performance of OCC with three criterions: classification accuracy, F-Score, and Area Under the Receiver Operating Characteristic Curve (AUC). For clarity, we briefly denote *pe* to represent classification accuracy.

In our experiments, the results are averaged over 20 random partitions of training and test datasets. The average values and standard deviations of three criterions are shown in Tables 4–9 for all datasets. The average values of each method shown in the Tables 3–8 are averaged over its best results on each partition, which correspond to the results with the optimal model parameters on each partition.

The best overall performance for each criterion is shown in bold italic. We observe that the best performance in each column, i.e., for each criterion, is obtained by the proposed method in most cases.

Table 6
Average values (%) and standard deviations (%) of three criterions for Landsat data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	79.72 ± 1.10	71.25 ± 0.91	91.54 ± 0.61
KOCSVM	85.98 ± 0.77	73.25 ± 1.88	90.88 ± 0.60
k-means	82.40 ± 1.03	73.76 ± 1.01	92.72 ± 1.13
PCA	71.28 ± 0.55	63.27 ± 0.61	80.11 ± 0.80
Gauss	81.77 ± 0.86	73.27 ± 0.89	$\boldsymbol{93.96 \pm 0.50}$
MST	73.26 ± 1.07	65.86 ± 0.78	87.25 ± 0.72
SOM	80.06 ± 2.49	70.29 ± 2.76	87.76 ± 2.07
Auto-encoder	86.01 ± 2.46	71.84 ± 7.42	92.35 ± 1.63
MPM	85.04 ± 1.21	69.95 ± 3.60	89.96 ± 0.97
LPDD	86.92 ± 1.36	74.54 ± 3.20	91.69 ± 0.77
IB-OCSVM	$\boldsymbol{88.06 \pm 0.86}$	$\boldsymbol{77.87 \pm 1.48}$	92.21 ± 0.80

Table 7
Average values (%) and standard deviations (%) of three criterions for Cleve data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	63.33 ± 2.52	70.57 ± 2.59	64.04 ± 3.73
KOCSVM	64.27 ± 2.39	70.88 ± 3.42	63.98 ± 3.59
k-means	68.33 ± 2.20	$\boldsymbol{72.82 \pm 2.23}$	76.97 ± 2.47
PCA	62.08 ± 1.33	68.24 ± 1.90	66.75 ± 3.86
Gauss	66.30 ± 2.17	72.18 2.49	76.20 ± 2.57
MST	61.51 ± 2.33	70.27 ± 1.83	75.97 ± 2.54
SOM	67.40 ± 3.64	65.16 ± 4.28	75.22 3.26
Auto-encoder	67.03 ± 4.25	62.69 ± 9.25	73.54 ± 5.31
MPM	72.60 ± 2.55	70.89 ± 3.28	79.34 ± 2.34
LPDD	70.63 ± 2.54	65.85 ± 5.73	77.38 ± 3.32
IB-OCSVM	$\boldsymbol{73.54 \pm 2.47}$	70.24 ± 4.28	$\boldsymbol{79.47 \pm 2.34}$

Table 8
Average values (%) and standard deviations (%) of three criterions for Australian data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVD	55.41 ± 2.86	66.67 ± 1.25	74.42 ± 4.19
KOCSVM	70.76 ± 5.37	67.44 ± 7.19	73.84 ± 4.94
k-means	69.29 ± 7.66	74.69 ± 4.33	82.53 ± 2.41
PCA	66.06 ± 3.83	72.13 ± 2.43	78.32 ± 4.35
Gauss	63.15 ± 4.39	70.66 ± 2.17	81.11 ± 2.40
MST	56.41 ± 2.48	67.99 ± 1.47	66.23 ± 2.79
SOM	61.14 ± 3.77	67.33 ± 3.07	67.84 ± 4.34
Auto-encoder	78.61 ± 6.15	77.65 ± 8.86	$\boldsymbol{83.14 \pm 4.60}$
MPM	81.90 ± 4.39	75.18 ± 2.48	80.73 ± 2.31
LPDD	67.99 ± 3.14	67.54 ± 3.52	71.73 ± 3.61
IB-OCSVM	$\boldsymbol{81.41 \pm 1.29}$	$\boldsymbol{80.36 \pm 1.87}$	82.57 ± 2.26

Table 9
Average values (%) and standard deviations (%) of three criterions for Breast_ cancer data over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	54.13 ± 1.69	66.15 ± 1.61	56.97 ± 2.92
KOCSVM	64.97 ± 1.91	65.59 ± 4.46	67.16 ± 2.58
k-means	63.63 ± 1.36	70.16 ± 1.82	67.98 ± 2.36
PCA	60.78 ± 1.48	68.53 ± 1.78	67.31 ± 2.96
Gauss	63.76 ± 1.39	70.45 ± 1.61	67.55 ± 2.16
MST	58.82 ± 1.91	69.58 ± 1.21	65.76 ± 1.90
SOM	61.86 ± 2.74	64.15 ± 3.42	66.90 ± 2.32
Auto-encoder	$\mathbf{66.74 \pm 2.09}$	68.12 ± 3.01	69.15 ± 3.12
MPM	65.03 ± 2.18	$\mathbf{69.29 \pm 2.44}$	68.57 ± 2.45
LPDD	66.52 ± 2.26	68.65 ± 3.57	69.69 ± 2.59
IB-OCSVM	66.60 ± 3.51	67.55 ± 3.50	$\mathbf{69.77 \pm 2.19}$

Table 10
Average values (%) and standard deviations (%) of three criterions for Sandia MiniSAR dataset over the 20 random partitions for the training and test sets.

Method	*pe*	F-Score	AUC
L-SVDD	82.14 ± 1.48	85.05 ± 1.45	87.30 ± 0.90
KOCSVM	84.72 ± 0.73	87.86 ± 0.54	88.18 ± 0.89
k-means	84.82 ± 1.44	86.97 ± 1.41	88.46 ± 1.25
PCA	73.83 ± 2.59	79.11 ± 2.31	77.99 ± 2.07
Gauss	85.48 ± 1.60	87.67 ± 1.57	$\boldsymbol{89.92 \pm 0.96}$
MST	84.13 ± 1.20	87.05 ± 1.04	89.08 ± 1.21
SOM	80.23 ± 2.18	82.31 ± 2.36	89.05 ± 1.29
Auto-encoder	83.42 ± 1.63	85.98 ± 1.50	87.65 ± 1.14
MPM	84.67 ± 1.82	86.85 ± 1.80	89.08 ± 1.52
LPDD	83.24 ± 2.58	85.11 ± 2.62	88.36 ± 1.31
IB-OCSVM	$\boldsymbol{85.77 \pm 0.83}$	$\boldsymbol{88.50 \pm 4.50}$	89.35 ± 0.69

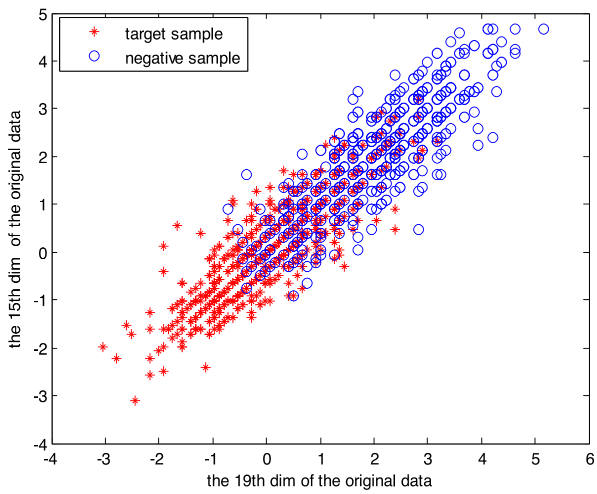
For the criterion of classification accuracy, IB-OCSVM achieves the highest score in four out of six datasets (i.e., Landsat, Pendigits, Waveform and Cleve), which demonstrates the good classification performance of IB-OCSVM. In four out of six datasets (i.e., Landsat, Pendigits, Waveform and Australian), IB-OCSVM outperforms all other methods for the criterion of F-Score, which proves the good balance between precision and recall of the proposed method. We achieve the highest or sub-highest AUC in most cases (i.e., Waveform, Cleve, Australian and Breast_cancer), which shows good generalization of IB-OCSVM. Therefore, results on Tables 3–8 show the effectiveness of the proposed method.

To explain the performance of IB-OCSVM further, we give the 2D feature distributions of the transformed features and the original features. Here, we choose the Waveform and Landsat datasets as the examples, and fix the model parameters $\eta = 0.2$ and $\sigma = 3$. The top four scored features are chosen based on LDA. Fig. 9(a), (b) show the original 2D feature distribution of the Waveform dataset; the corresponding transformed feature distribution is depicted in Fig. 9(c) and (d). The original feature distribution of the Landsat dataset is represented in Fig. 10(a), (b); Fig. 10(c), (d) show the corresponding transformed feature distribution. It is apparent that the original features mix with each other; while the transformed features are more separable than the original ones.

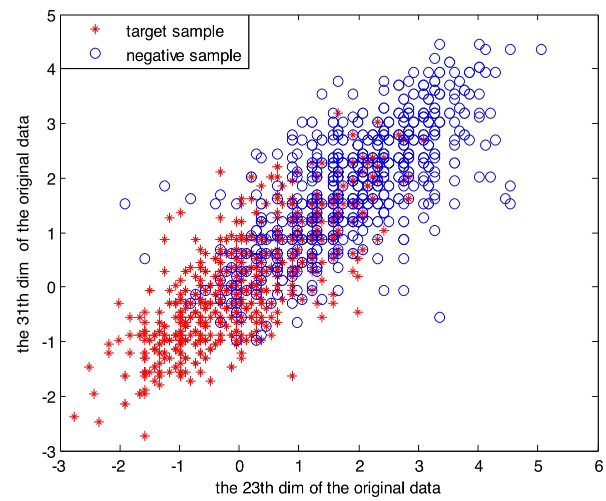
4.3. Results on synthetic aperture radar (SAR) real data

In Sections 4.1 and 4.2, we present the performance of IB-OCSVM on the synthetic dataset and benchmark datasets. Now, the performance of IB-OCSVM on the SAR real data is shown in this Section. The SAR real data come from Sandia MiniSAR dataset.[2] There are totally nine SAR images in this dataset. Fig. 11 shows a SAR image example in Sandia MiniSAR dataset. With the constant false alarm rate (CFAR) detection algorithm, we obtain 346 patches, including 248 target patches and 98 non-target patches, from SAR images in Sandia MiniSAR dataset, and some patch examples are shown in Fig. 12. We set the patches only including the car as positive samples, and the remaining patches as negative samples. In Fig. 12, the first row corresponds to positive samples, and the second row corresponds to negative samples.

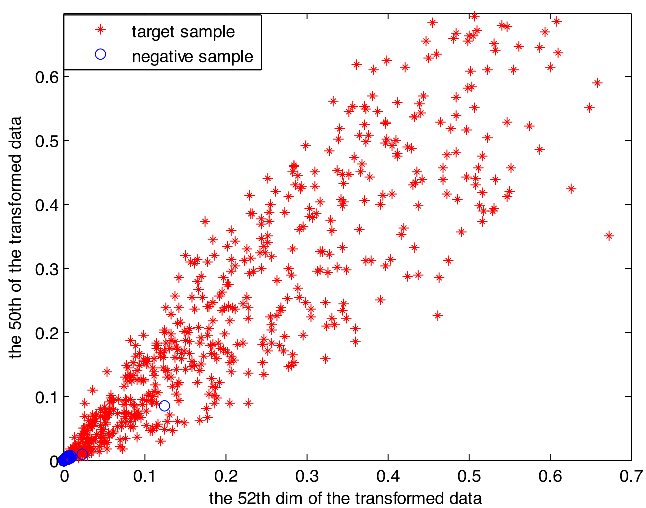
In our experiments, the number of training samples is 150, and the number of test samples is 198. Same with the experiments on benchmark dataset, the results is averaged over 20 random partitions of training and test datasets. The performances, represented by the average values and standard deviations of the three criterions, are shown in Table 10. The model parameters' settings are the same as those in the experiments on benchmark datasets.



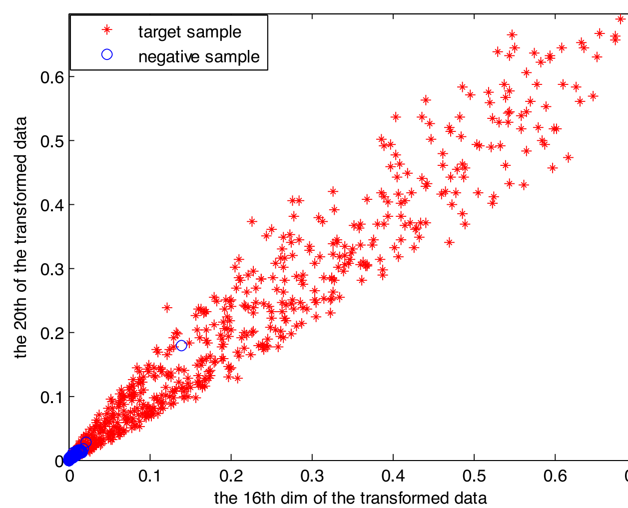
(a) 19 *th* versus 15 *th* for the original features;



(b) 23 *th* versus 31 *th* for the original features



(c) 52 *th* versus 50 *th* for the transformed features;



(d) 16 *th* versus 20 *th* for the transformed features

Fig 10. The 2D feature distribution of original data and transformed data on Landsat dataset.

Fig 11. A SAR image example in Sandia MiniSAR dataset.

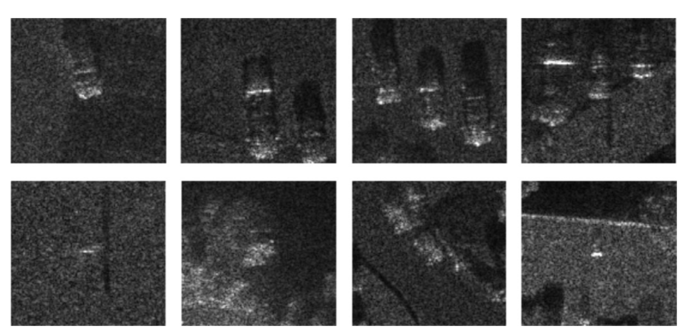
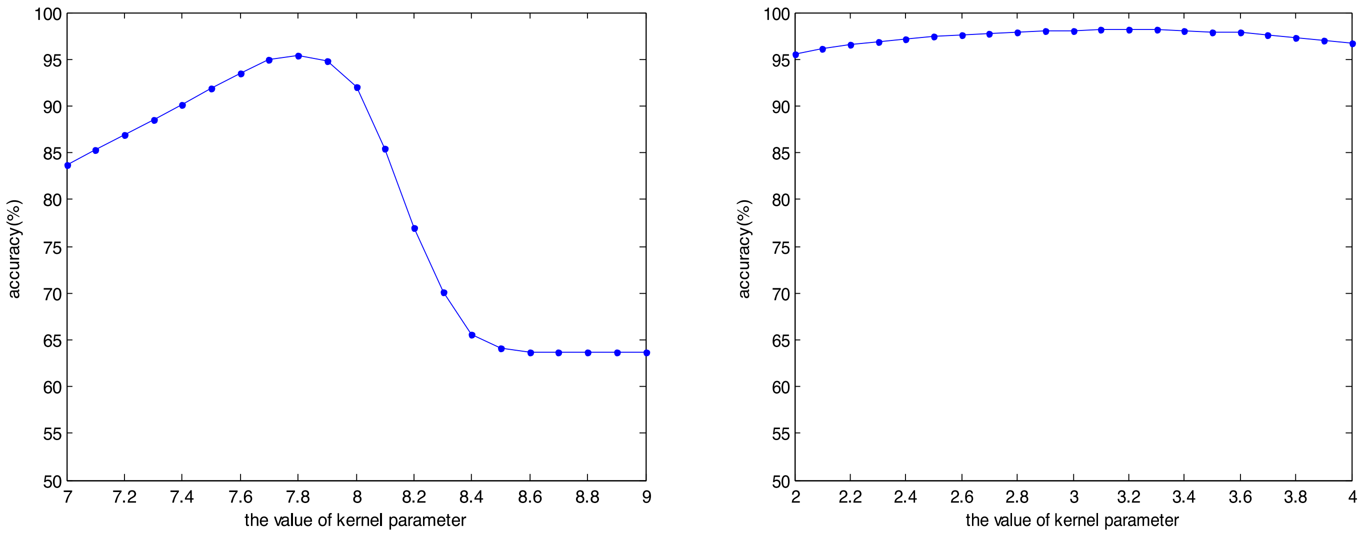
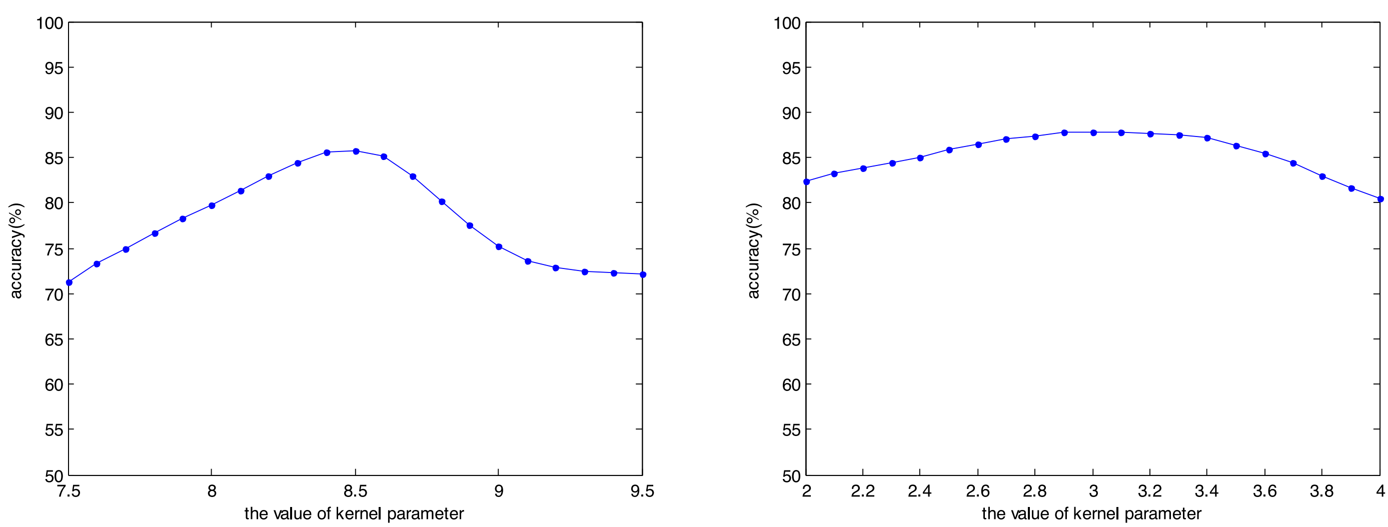


Fig. 12. Some patch examples from SAR images in Sandia MiniSAR dataset, where the first row corresponds to the positive samples, and the second row corresponds to the negative samples.



(a) KOCSVM; (b) IB-OCSVM

Fig 13. The classification accuracies of the KOCSVM and IB-OCSVM with different values of kernel parameter on Pendigits dataset.



(a) KOCSVM; (b) IB-OCSVM

Fig. 14. The classification accuracies of the KOCSVM and IB-OCSVM with different values of kernel parameter on Landsat dataset.

Looking at the results above, it is clear that IB-OCSVM achieves the highest *pe* and F-score, and gets the sub-optimal result for the criterion of AUC, which shows that IB-OCSVM not only can achieve good classification performance, but also has good generalization.

4.4. Analysis of model parameters

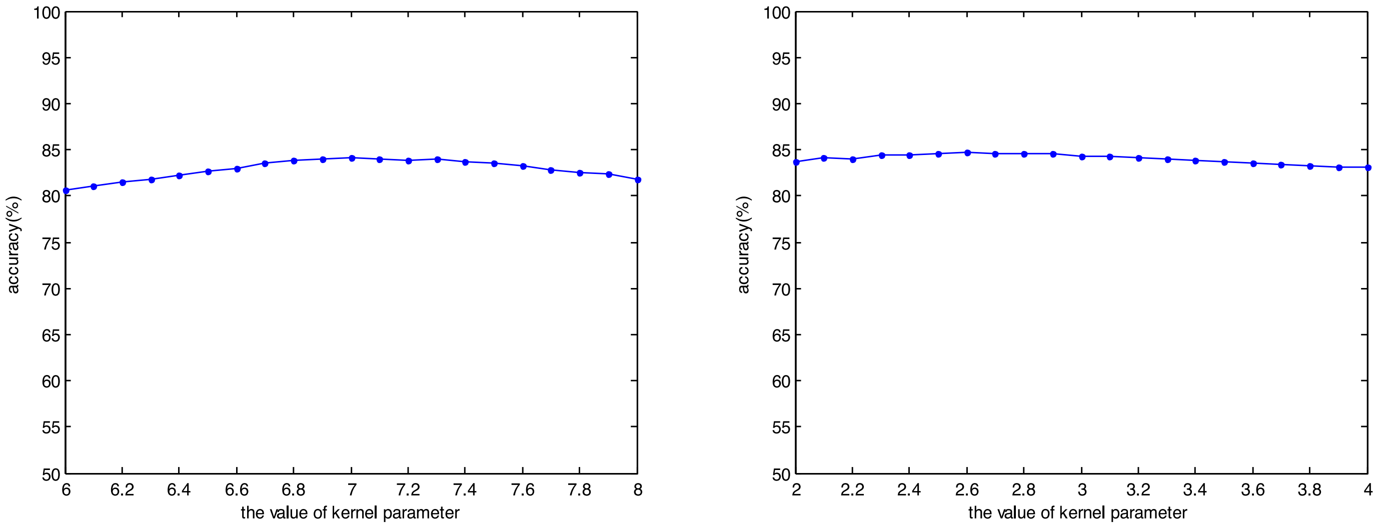
In Sections 4.2 and 4.3, the average values and standard deviations of three criterions on benchmark datasets and SAR real data is given. Now, we make an analysis on the robustness of our IB-OCSVM to the model parameters in this Section.

4.4.1. Gaussian Kernel parameter

Firstly, the influence of Gaussian kernel parameter on IB-OCSVM and KOCSVM is given. The value of Gaussian kernel parameter σ varies from $\sigma_{opt} - 1$ to $\sigma_{opt} + 1$ at intervals of 0.1 for both methods, where σ_{opt} represents its optimal value. For IB-OCSVM, we fix the model parameter $\eta = 0.2$ when we analyze the robustness to the Gaussian kernel parameter. We choose Pendigits dataset, Landsat dataset and SAR real data as the examples. The classification accuracies on different datasets with different values of the Gaussian kernel parameter for KOCSVM and IB-OCSVM are given in Figs. 13–15.

To quantitatively compare the robustness of two methods, Table 11 further gives the average classification accuracies and standard deviations of two methods on three datasets when the value of kernel parameter varies. Obviously, the proposed method,

[2] http://www.sandia.gov/radar/imagery.



(a) KOCSVM; (b) IB-OCSVM

Fig 15. The classification accuracies of the KOCSVM and IB-OCSVM with different values of kernel parameter on SAR real dataset.

Table 11
The average classification accuracies and standard deviations of the KOCSVM and IB-OCSVM in Figs. 13–15.

Method	Pendigits dataset	Landsat dataset	SAR real dataset
KOCSVM	79.90 ± 12.94	78.03 ± 5.05	82.87 ± 1.05
IB-OCSVM	**97.43 ± 0.74**	**85.39 ± 2.23**	**84.03 ± 0.52**

IB-OCSVM, yields the highest classification accuracies and the lowest deviations on three datasets.

When the kernel parameter varies in a certain range, the classification accuracies of the IB-OCSVM on all of the three datasets change a little, while the classification results of the KOCSVM change greatly. Compared with the KOCSVM, the IB-OCSVM is more robust to the Gaussian kernel parameter used in the transformation.

4.4.2. The ratio of the number of base vectors to that of the training data η

Now, we make an analysis on the model parameter η. Same as the Gaussian kernel parameter, we present classification accuracies on Pendigits, Landsat and SAR real datasets with different values of η in Fig. 16. The value of the Gaussian kernel parameter is set to be 3 when we analyze the influence of η.

In Fig. 16, the classification performance of IB-OCSVM is worse when the value of η is too small, e.g., $\eta \leq 0.1$. In addition, when the value of η is too big, e.g., $\eta \geq 0.8$, the classification accuracy also decreases. When the value of η is too small, the number of the samples selected from each cluster is small, thus the base vector set can not represent the structure-distribution of training data. When the value of η is too large, the dimension of transformed features is too high, which may lead to the curse of dimensionality and overfitting. In general, IB-OCSVM yields a good performance with $0.15 \leq \eta \leq 0.3$. Therefore, we usually set $\eta = 0.2$.

4.4.3. Parameters used in Gibbs sampling

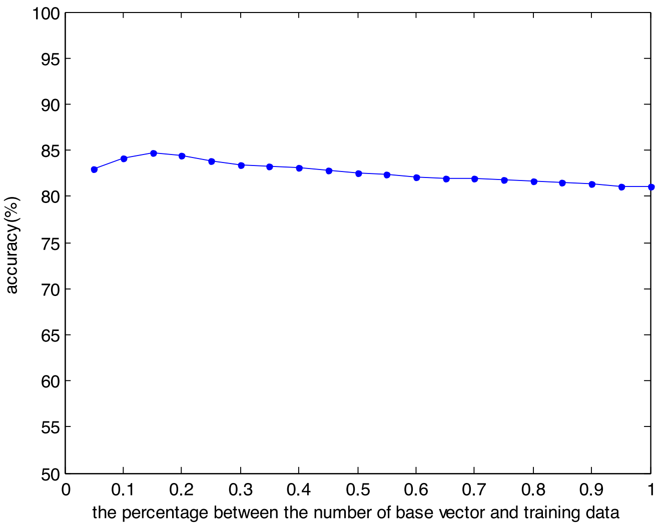
As discussed in [45,46], when the Gibbs sampling is used to estimate a posterior distribution, a key issue is to determine when the procedure has essentially converged. In order to investigate the convergence of the Gibbs sampling in our model, we give the classification accuracies with different values of burn-in parameter $I_{burn-in}$, collection parameter I_{num} and sampling interval I_{space}, respectively. Fig. 17(a)–(g) show the classification accuracy of our method with different values of $I_{burn-in}$ for UCI benchmark datasets and real SAR dataset, where 300 samples are collected with the interval of ten. We can see from Fig. 17 that the performance has been quite stable when the number of burn-in samples is larger than 2500. The standard deviation of classification accuracies for the final 500 iterations in the burn-in step for each dataset is given in Table 12. The values in Table 12 approximate to zero. Therefore, the training procedure has converged through 2500 iterations in the burn-in step, and we set the value of $I_{burn-in}$ to be 3000 in our experiments.

In Fig. 18(a)–(g), we present the classification accuracies of our method with different values of I_{num} for UCI benchmark datasets and real SAR dataset, where the value of $I_{burn-in}$ is set to be 3000 and the sampling interval is set to be 10. It is apparent that when the number of collection samples is larger than 250, the classification accuracies have converged to stable values. Table 13 shows the standard deviation of classification accuracies for the final 50 iterations in the collection procedure for each dataset. The values in Table 12 get close to zero. Therefore, the IB-OCSVM algorithm has converged when the value of I_{num} is larger than 250, and we set the value of I_{num} to be 300 in our experiments.

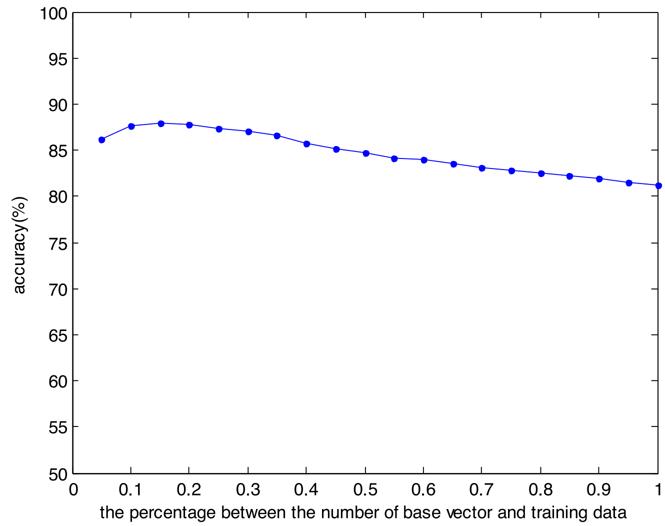
In the Gibbs sampling, the sampling interval I_{space} is a constant larger than 1 to eliminate the correlation of adjacent collection samples. The classification accuracies of our method with different values of I_{space} for two UCI datasets is depicted in Fig. 19, where the iterations of burn-in step are 3000 and 300 samples are collected. Table 14 shows the standard deviations of classification accuracies in Fig. 19(a), (b). It is apparent that the value of I_{space} has little influence on the classification accuracy. Therefore, the sampling interval I_{space} is set to be 10 in our experiments.

4.4.4. Truncation level C for the number of clusters

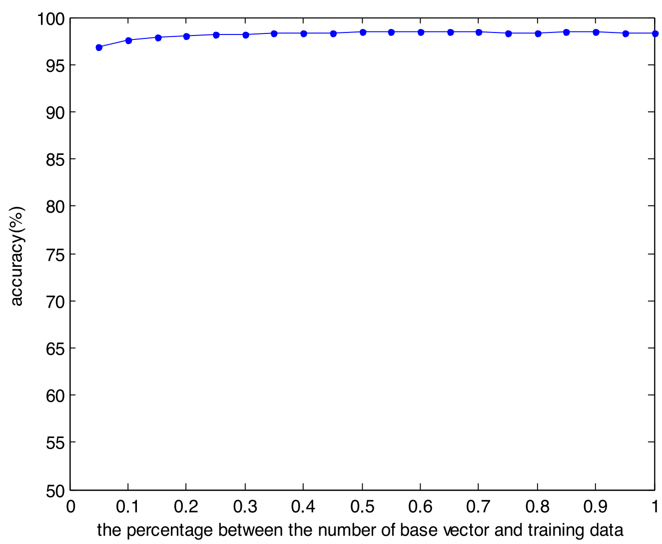
In general, the truncation level C can be set as any value lager than the real number of clusters of the dataset. In order to explore the real number of clusters of the dataset, we set the value of C as 20. Fig. 20(a)–(g) show the number of samples belonging to each



(a) SAR real data;



(b) Landsat dataset



(c) Pendigits dataset

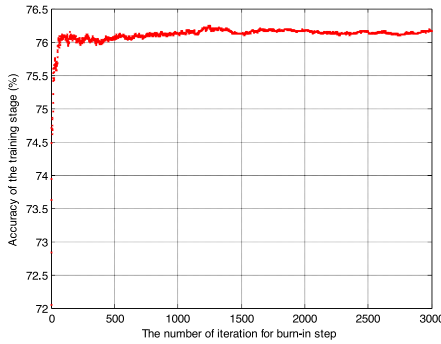
Fig. 16. Classification accuracies of the IB-OCSVM with different values of η on SAR real data, Landsat dataset and Pendigits dataset.

Table 12
The standard deviation of classification accuracies for the final 500 iterations in the burn-in step for each dataset.

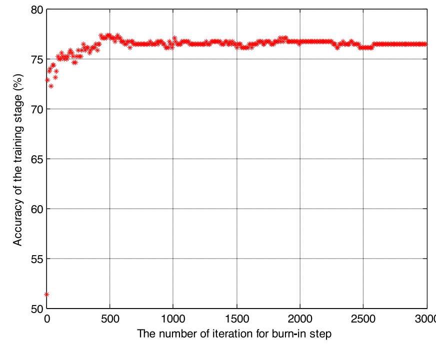
Dateset	Australian	Breast_cancer	Cleve	Landsat	Pendigits	Waveform	Minisar
Standard deviation	2.5328×10^{-4}	2.5537×10^{-4}	3.4496×10^{-4}	1.7681×10^{-4}	5.2288×10^{-5}	6.0654×10^{-4}	4.4453×10^{-4}

Table 13
The standard deviation of classification accuracies for the final 50 iterations in the collection procedure for each dataset.

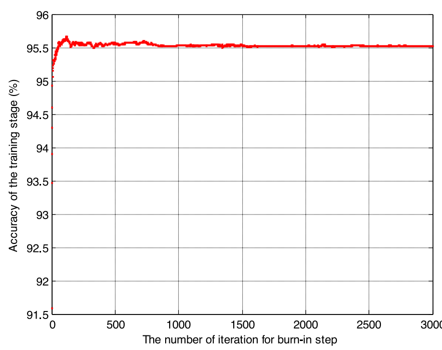
Dateset	Australian	Breast_cancer	Cleve	Landsat	Pendigits	Waveform	Minisar
Standard deviation	3.7593×10^{-4}	4.7324×10^{-4}	3.7954×10^{-4}	1.3460×10^{-4}	3.9244×10^{-5}	2.0319×10^{-4}	3.7411×10^{-4}



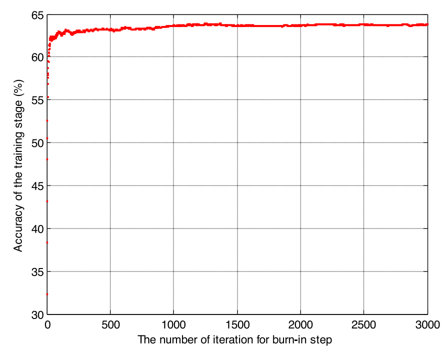
(a). Landsat dataset;



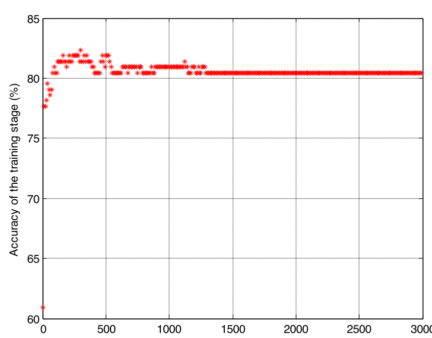
(b) Waveform dataset



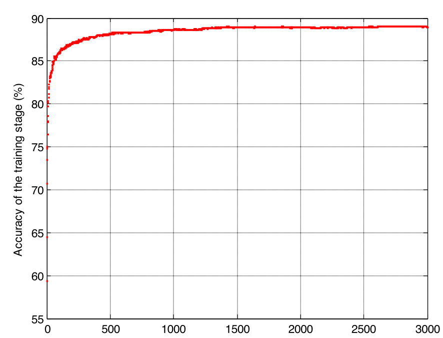
(c). Pendigits dataset;



(d). Cleve dataset



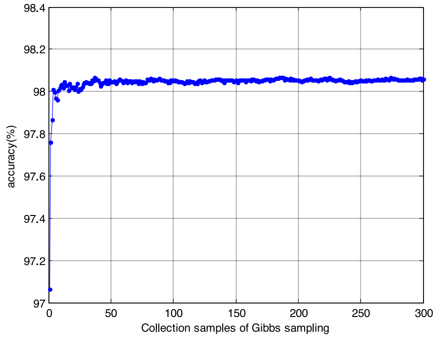
(e). Australian dataset;



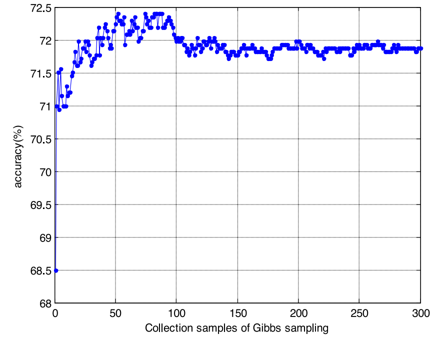
(f). Breast_cancer dataset

(g) Minisar dataset

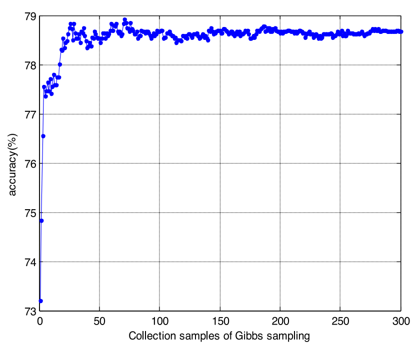
Fig 17. The classification performance with different numbers of burn-in samples for UCI datasets and real SAR dataset.



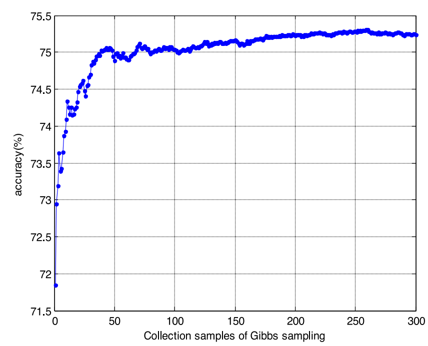
(a) Pendigits dataset;



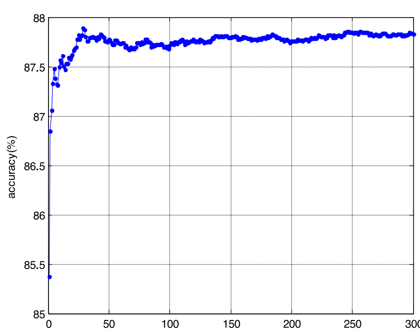
(b) Cleve dataset



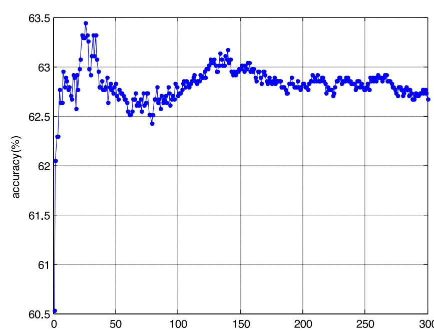
(c) Australian dataset;



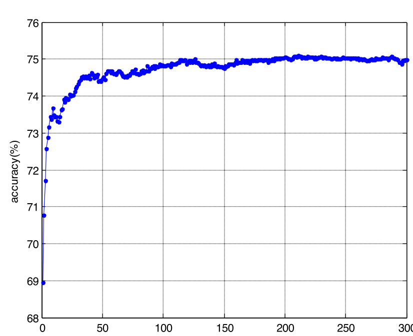
(d) Waveform dataset



(e) Landsat dataset;

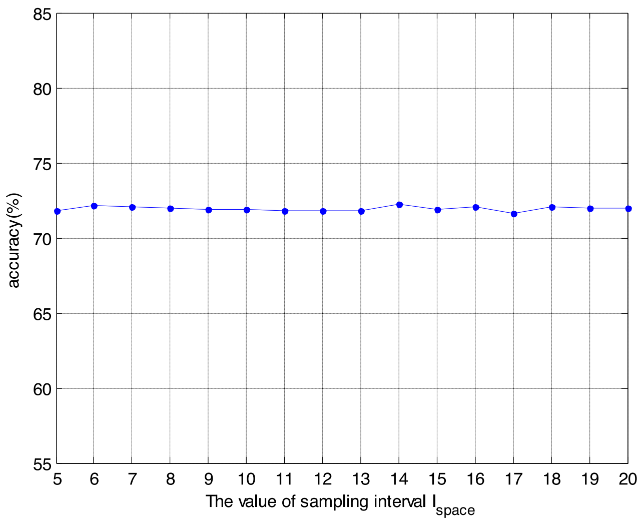


(f) Breast_cancer dataset

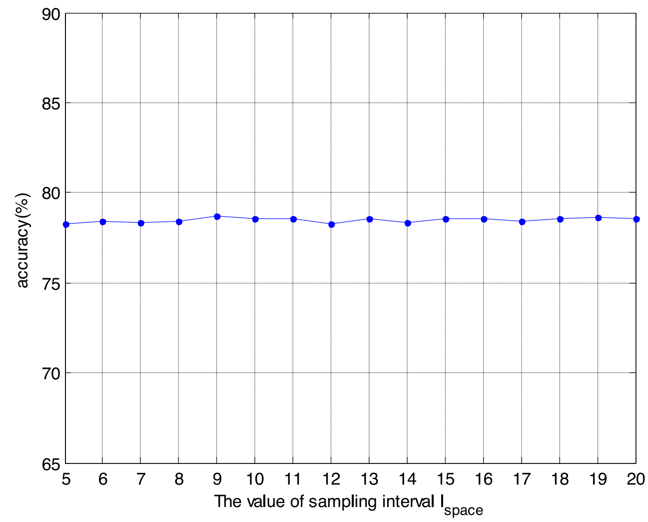


(g) Minisar dataset

Fig 18. The classification performance with different numbers of collection samples for UCI datasets and real SAR dataset.



(a) Cleve dataset;



(b) Australian dataset

Fig 19. The classification performance with the value of sampling interval for Cleve dataset and Australian dataset.

Table 14
The standard deviations of classification accuracies in Figs. 19(a) and (b).

Dataset	Cleve	Australian
Standard deviation	0.0015	0.0013

cluster for a sampling in the collection step for all datasets used in our paper. It can be seen from Eq. (20) that the real numbers of clusters are smaller than 10 for all datasets. Therefore, we set the truncation level C to be 10 in our experiments.

4.4.5. Constant/Penalty parameter l

Here we give some explanation about the value of parameter l. When the value of l is too small, the model may be underfitting; when the value of l is too large, the model may be overfitting. We make an analysis on the influence of this parameter for all datasets used in the paper. In Fig. 21(a)–(g), we show the classification performance with the different values of l for each dataset. We can see from Fig. 21 that our method obtains the best performance when the value of l is chosen as 0.1. Therefore, we set the value of parameter l to be 0.1 in our paper.

The parameter η can be given as $\eta = l/2N$. We can see that there is a consistent one-by-one match between the parameter l and the parameter η as N is fixed. Therefore, we only give the empirical investigation of the parameter l.

4.4.6. Hyperplane parameter ρ

In our method, parameter $\mathbf{w}$ is learned from the training data, while parameter ρ is set as a constant. If the value of parameter ρ is set too large, the distance between the origin and the hyperplane is far, which will lead to misclassify some positive samples. If the value of parameter ρ is set too small, the distance between the origin and the hyperplane is close, which will lead to misclassify some negative samples.

In order to explore the influence of the parameter ρ, we give the classification accuracies of our method with different values of ρ for UCI benchmark datasets and real SAR dataset in Fig. 22. We can see from Fig. 22 that our method obtains good performance in most datasets when the value of ρ is chosen as 1. Therefore, we set the value of parameter ρ to be 1 in our paper.

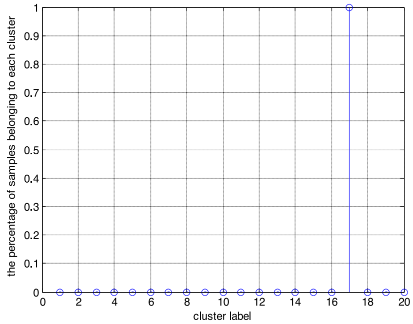
Table 15
Computation time (s) of the one class classification methods on Cleve and Landsat datasets.

Method	Cleve dataset		Landsat dataset	
	Training time	Test time	Training time	Test time
L-SVDD	1.006092	0.122675	1.571878	0.411332
KOCSVM	1.213745	0.132368	1.822413	0.512742
k-means	0.283226	0.107473	0.362883	0.119516
PCA	0.50177	***0.078318***	0.51205	***0.100478***
Gaussian	***0.240656***	0.101783	***0.270102***	0.117353
MST	0.817947	0.130193	2.934027	3.349392
SOM	27.782627	0.10095	122.611088	0.109035
Auto-encoder	19.980187	16.877965	530.921214	22.361604
MPM	0.671223	0.124465	1.17576	0.232429
LPDD	1.26046	0.403607	10.177256	7.452492
IB-OCSVM	93.120319	1.896424	371.16457	34.446795

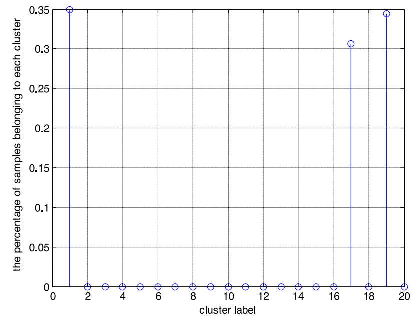
4.5. Computation time

In this Section, we compare the one class classification methods from the training and test time points of view. Here we select Cleve and Landsat datasets as two examples. Table 15 shows the computation time of the one class classification methods on these two datasets. The best overall performance for each column is marked in bold italic.

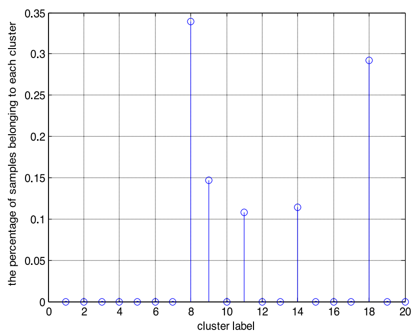
Compared with Cleve dataset, the number of the training data in Landset dataset is larger and its original dimension is higher. Therefore, the computation time of the one-class classification methods on Landsat dataset is longer than that on Cleve dataset. In the training step, the computation time costs of the methods, i.e., L-SVDD, KOCSVM, PCA, $k - means$, Gaussian, MST, MPM, LPDD, are less than those of Auto-encoder, SOM and our method, and the computation burden of our method is in the same order of magnitude as Auto-encoder and SOM. In the test step, the computation time costs of Auto-encoder and our methods are in the same order of magnitude, and more than those of the rest compared methods. Since Gibbs sampler is used to estimate posterior distribution in IB-OCSVM, the computation burden of our method is larger than that of some compared methods. This is a disadvantage of our work. In the future work, we will try to use variational Bayesian (VB) to estimate the model parameters to shorten the test time.



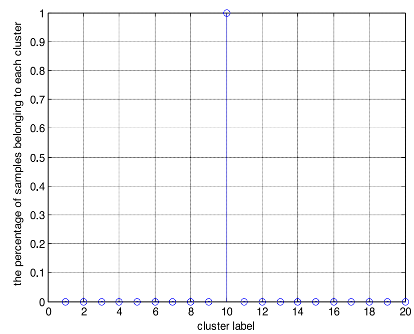
(a) Waveform dataset;



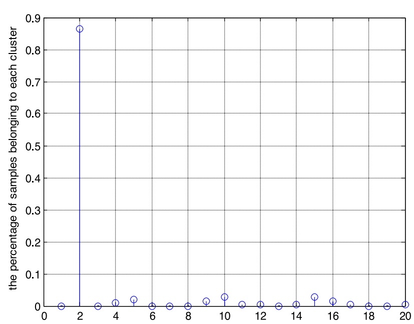
(b) Landsat dataset



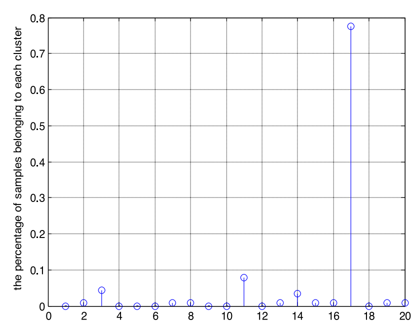
(c) Pendigits dataset;



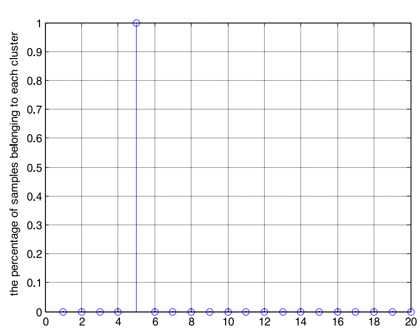
(d) Cleve dataset



(e) Australian dataset;

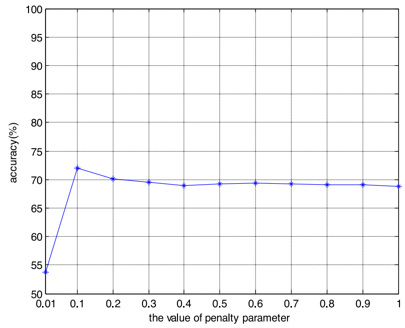


(f) Breast_cancer dataset

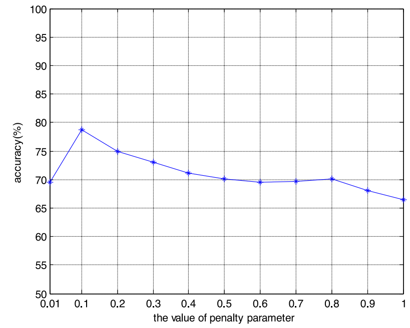


(g) Minisar dataset

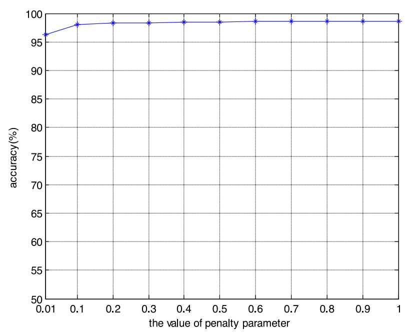
Fig 20. The number of samples belonging to each cluster for a sampling in the collection step for UCI datasets and real SAR dataset.



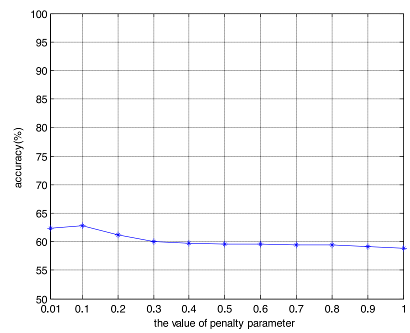
(a) Cleve dataset;



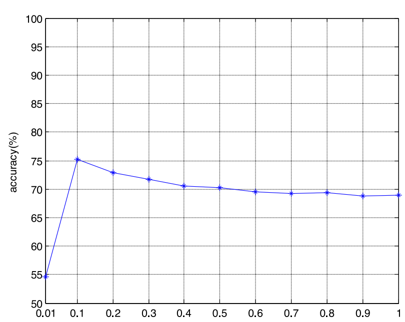
(b) Australian dataset



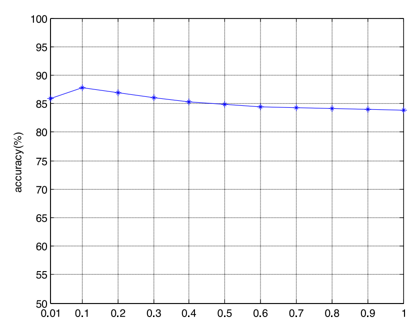
(c) Pendigits dataset;



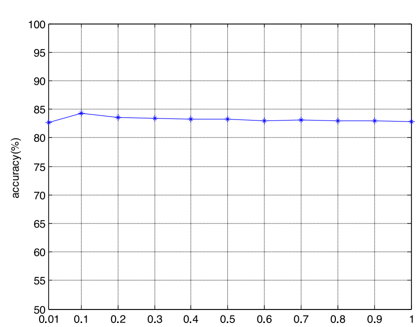
(d) Breast_cancer dataset



(e) Waveform dataset;

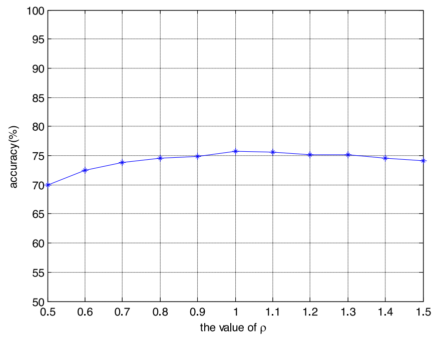


(f) Landsat dataset

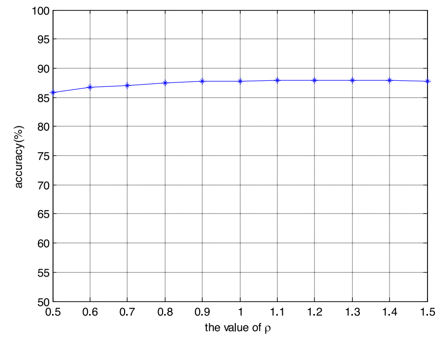


(g) Real SAR dataset

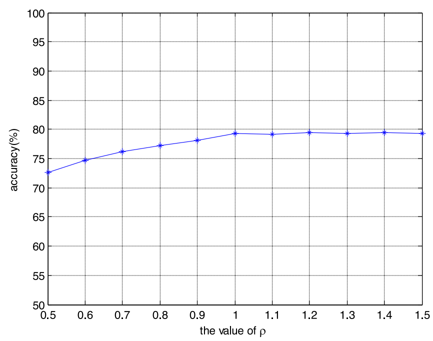
Fig 21. The classification performance with the value of parameter *l* for six UCI datasets and real SAR dataset.



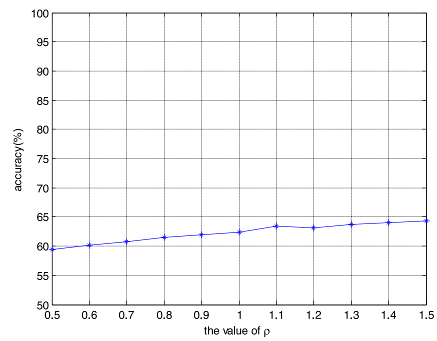
(a) Waveform dataset;



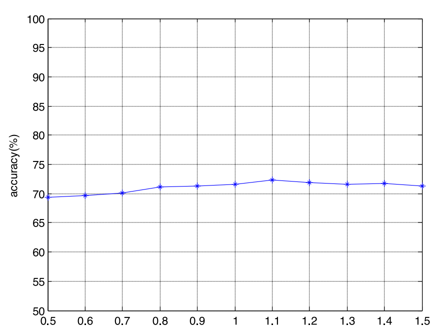
(b) Landsat dataset



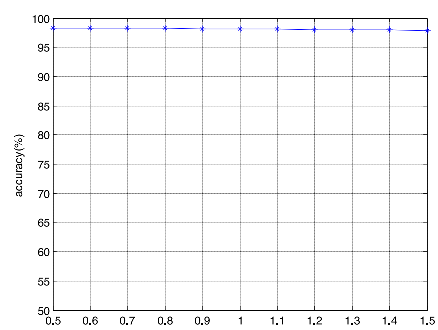
(c) Australian dataset;



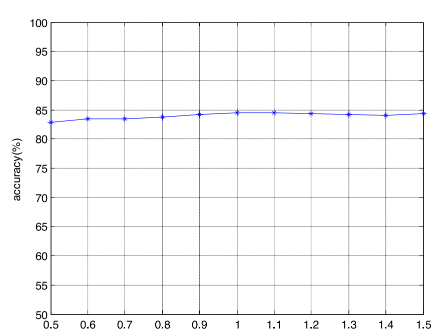
(d) Breast_cancer dataset



(e) Cleve dataset;



(f) Pendigits dataset



(g) Minisar dataset

Fig. 22. The classification performance with different values of parameter ρ for UCI datasets and real SAR dataset.

5. Conclusion

This study proposes a method, namely, IB-OCSVM, to reduce the sensitivity of OCC to the model parameter. The IB-OSVM is a combination of the reconstruction-based and boundary-based methods. In the reconstruction step, we cluster training data with DPM. In the boundary construction step, we use the modified OCSVM, which uses a new transformation method rather than the original kernel transformation to construct a linear classification hyperplane in each cluster. The DPM and the modified OCSVM link up with each other through joint optimization, which is achieved by a unified Bayesian frame. Compared with the traditional OCSVM that requires all kernel transformed features to be linearly separable, the IB-OCSVM only requires the linear separability in each cluster in the feature space. Therefore, our model is more robust to the model parameter used in the new feature transformation. The experimental results on the synthetic dataset, benchmark datasets, and SAR real data show that the IB-OCSVM is more robust to the model parameter and obtains better classification performance than the traditional methods. Furthermore, the 2D feature distribution graphs in the experiments clearly illustrate that the proposed transformation method is feasible.

A shortcoming of MCMC is that each collected state is required to be collected, which not only limits its applicability to Big Data applications, but also limits its applicability to real-time classification. To solve the memory problem in Big Data applications, one possible solution is to use an online version of MCMC, namely stochastic gradient MCMC (SGMCMC) [51]. Compared with MCMC, the variational Bayesian (VB) inference can improve the test speed. In our future work, we may try these inference methods. In addition, with the Bayesian solution to the optimization problem of the modified OCSVM provided in this study, we have the chance to solve the optimization problem of the Gaussian kernel parameter directly with the Bayesian method, and this will be studied in our future work.

Acknowledgment

This work was partially supported by the National Science Foundation of China (No. 61771362 and No. 61671354), and the Science Foundation of Shaanxi Province (No. 2015JZ016).

References

[1] D.M.J. Tax, One-class classification: concept-learning in the absence of counter-examples (Ph. D. thesis), Delft University of Technology, Stevinweg, The Netherlands, 2001.
[2] M.A.F. Pimentel, D.A. Clifton, L. Clifton, et al., A review of novelty detection, Signal Process. 99 (2014) 215–249.
[3] M. Kemmler, E. Rodner, J. Denzler, One-class classification with Gaussian processes, Pattern Recogn. 46 (12) (2013) 3507–3518.
[4] D.M.J. Tax, R.P.W. Duin, Support vector domain description, Pattern Recogn. Lett. 20 (11–13) (1999) 1191–1199.
[5] P. Juszczak, D.M.J. Tax, E. PȩKalska, et al., Minimum spanning tree based one–class classifier, Neurocomputing 72 (7–9) (2009) 1859–1869.
[6] E.D. Santis, L. Livi, A. Sadeghian, et al., Modeling and recognition of smart grid faults by a combined approach of dissimilarity learning and one-class classification, Neurocomputing 170 (2014) 368–383.
[7] D.M.J. Tax, R.P.W. Duin, Uniform object generation for optimizing one–class classifiers, J. Mach. Learn. Res. 2 (2) (2002) 155–173.
[8] G.L. Bosco, L. Pinello, in: A Fuzzy One Class Classifier for Multi Layer Model Fuzzy Logic and Applications, Springer, Berlin Heidelberg, 2009, pp. 124–131.
[9] E. Pekalska, D.M.J. Tax, R.P.W. Duin, One-class LP classifiers for dissimilarity representations, Adv. Neural Inf. Process. Syst (2002) 761–768.
[10] C.D. Wang, J.H. Lai, Position regularized support vector domain description, Pattern Recogn. 46 (3) (2013) 875–884.
[11] P Bodesheim, A Freytag, E Rodner, et al., Kernel null space methods for novelty detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2013, pp. 3374–3381.
[12] F. Angiulli, Prototype-based domain description for one-class classification, IEEE Trans. Pattern Anal. Mach. Intell. 34 (6) (2012) 1131–1144.
[13] K.Y. Lee, D.W. Kim, K.H. Lee, et al., Density-induced support vector data description, IEEE Trans. Neural Netw. 18 (1) (2007) 284–289.
[14] X. Ding, Y. Li, A Belatreche, et al., An experimental evaluation of novelty detection methods, Neurocomputing 135 (2014) 313–327.
[15] H. Izakian, W. Pedrycz, Anomaly detection and characterization in spatial time series data: a cluster-centric approach, IEEE Trans. Fuzzy Syst. 22 (6) (2014) 1612–1624.
[16] M. Bicego, M.A.T. Figueiredo, Soft clustering using weighted one-class support vector machines, Pattern Recogn. 42 (1) (2009) 27–32.
[17] L. Livi, A. Sadeghian, W. Pedrycz, Entropic one-class classifiers, IEEE Trans. Neural Netw. Learn. Syst. 26 (12) (2015) 3187–3200.
[18] F. Dufrenois, J.C. Noyer, One class proximal support vector machines, Pattern Recogn. 52 (2016) 96–112.
[19] C. Bishop, Pattern Recognition and Machine Learning (Information Science and Statistics), 1st edn., Springer, New York, 2006 corr. 2nd printing edn 2007.
[20] G.J. McLachlan, K.E. Basford, in: Mixture models. Inference and Applications to clustering Statistics: Textbooks and Monographs, 1988, Dekker, New York, 1988, p. 1.
[21] Y. Agusta, D.L. Dowe, Unsupervised learning of gamma mixture models using minimum message length, in: Proceedings of the 3rd IASTED Conference on Artificial Intelligence and Applications, 2003, pp. 457–462.
[22] I. Mayrose, N Friedman, T. Pupko, A Gamma mixture model better accounts for among site rate heterogeneity, Bioinformatics 21 (Suppl 2) (2005) ii151–ii158.
[23] A.X. Carvalho, M.A. Tanner, Modelling nonlinear count time series with local mixtures of Poisson autoregressions, Comput. Stat. Data Anal. 51 (11) (2007) 5266–5294.
[24] M. Svensén, C.M. Bishop, Robust Bayesian mixture modelling, Neurocomputing 64 (2005) 235–252.
[25] A. Stranjak, P.S. Dutta, M. Ebden, et al., A multi-agent simulation system for prediction and scheduling of aero engine overhaul, in: Proceedings of the 7th International Joint Conference on Autonomous Agents and Multiagent Systems: Industrial Track. International Foundation for Autonomous Agents and Multiagent Systems, 2008, pp. 81–88.
[26] B. Schölkopf, R. Williamson, A. Smola, et al., SV estimation of a distribution's support, Adv. Neural Inf. Process. Syst. (1999) 12.
[27] L.E. Ghaoui, M.I. Jordan, G.R. Lanckriet, Robust novelty detection with single–class MPM, in: Proceedings of the Advances in Neural Information Processing Systems, 2002, pp. 905–912.
[28] C.M. Bishop, Neural Networks For Pattern recognition, Oxford University Press, 1995.
[29] V. Vapnik, The Nature of Statistical Learning theory, Springer Science & Business Media, 2013.
[30] V.N. Vapnik, An overview of statistical learning theory, IEEE Trans. Neural Netw. 10 (5) (1999) 988–999.
[31] J. Platt, Sequential minimal optimization: a fast algorithm for training support vector machines, 208, 1998, pp. 212–223.
[32] T.S. Ferguson, A Bayesian analysis of some nonparametric problems, Ann. Stat. 1 (1973) 209–230.
[33] J. Sethuraman, A constructive definition of Dirichlet priors, Stat. Sinica 4 (1994) 639–650.
[34] C.E. Antoniak, Mixtures of Dirichlet processes with applications to Bayesian nonparametric problems, Ann. Stat. 2 (1974) 1152–1174.
[35] D.M. Blei, M.I. Jordan, Variational inference for Dirichlet process mixtures, Bayesian Anal. 1 (1) (2006) 121–144.
[36] P. Sollich, Bayesian methods for support vector machines: evidence and predictive class probabilities, Mach. Learn. 46 (1–3) (2002) 21–52.
[37] N.G. Polson, S.L. Scott, Data augmentation for support vector machines, Bayesian Anal. 6 (1) (2011) 1–23.
[38] A.E. Gelfand, A.F.M. Smith, Sampling-based approaches to calculating marginal densities, J. Am. Stat. Assoc. 85 (410) (1990) 398–409.
[39] S. Geman, D. Geman, Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images, IEEE Trans. Pattern Anal. Mach. Intell. 6 (6) (1984) 721–741.
[40] D.B. Dunson, J.H. Park, Kernel stick-breaking processes, Biometrika 95 (2) (2008) 307–323.
[41] A. Gelman, J.B. Carlin, H.S. Stern, et al., Bayesian Data Analysis, Chapman & Hall/CRC, Boca Raton, FL, USA, 2014.
[42] R.A. Fisher, The use of multiple measurements in taxonomic problems, Ann. Eugen. 7 (2) (1936) 179–188.
[43] I. Jolliffe, Principal Component analysis, John Wiley & Sons, Ltd, 2002.
[44] T. Kohonen, P. Somervuo, Self-organizing maps of symbol strings, Neurocomputing 21 (1) (1998) 19–30.
[45] X. Zhang, B. Chen, H. Liu, et al., Infinite max-margin factor analysis via data augmentation, Pattern Recogn. 52 (C) (2016) 17–32.
[46] B. Chen, H. Zhang, X. Zhang, et al., Max-margin discriminant projection via data augmentation, IEEE Trans. Knowl. Data Eng. 27 (7) (2015). 1–1.
[47] H. Hoffmann, Kernel PCA for novelty detection, Pattern Recogn. 40 (3) (2007) 863–874.
[48] F. Dufrenois, A one-class Kernel Fisher criterion for outlier detection, IEEE Trans. Neural Netw. Learn. Syst. 26 (5) (2014) 982–994.
[49] Y.W. Teh, M.I. Jordan, M.J. Beal, et al., Hierarchical Dirichlet processes, J. Am. Stat. Assoc. 101 (476) (2006) 1566–1581.
[50] D. Reynolds, Gaussian mixture models, Encycl. Biom. 2 (2015) 827–832.
[51] Y.A. Ma, T Chen, E Fox, A complete recipe for stochastic gradient MCMC, in: Proceedings of Advances in Neural Information Processing Systems, 2015, pp. 2917–2925.

Wei Zhang received B.Eng.degree in electrical and information engineering from Xidian University, Xi'an China, in 2014. He is currently pursuing the Ph.D. degree in signal processing with the National Laboratory of Radar Signal Processing, Xidian University. His research interests include radar signal processing, radar automatic target recognition, and pattern recognition.

Lan Du received the B.S., M.S. and Ph.D. degrees in electronic engineering from Xidian University, Xi'an, China, in Jul. 2001, Mar. 2004 and Jun. 2007, respectively. Her doctoral dissertation was granted National Excellent Doctoral Dissertation of PR China in 2009. From Sep. 2007 to Sep. 2009, she did research work as a Postdoctoral Research Associate in the Department of Electrical and Computer Engineering, Duke University, Durham, NC 27,708 USA. She is currently a Professor at National Lab. of Radar Signal Processing, Xidian University. Her research work is supported by NSFC for Excellent Young Scholars. Her main research interests are in the fields of statistical signal processing and machine learning with application to radar target recognition.

Liling Li received the M.S. degree in electronic engineering from Xidian University, Xi'an, China, in Dec. 2014. Her research interests include pattern recognition and radar automatic target recognition (RATR).